Meeting 11. Neurons as Matrices

Act 0 Meeting 10, Revisited · Act 1 A Layer of Neurons Is a Matrix · Act 2 The Same Layer in PyTorch, and on a GPU

A **neuron** is a rule that converts a point of the data into one number, its **score**. These notes store a layer of neurons as a matrix, apply it to a whole batch of points with one matrix product, and turn the scores into probabilities. The same computation is then written in PyTorch and timed on two processors, a CPU and a GPU.

The meeting opened with a review of meeting 10 (Act 0). Act 1 developed two examples: four neurons that read the high and low temperature of a day and name its season, and five neurons that read the colour of a pixel and name the nearest of five reference colours. Act 2 moved the second layer to PyTorch and to a GPU. Each act ended with a Colab block of the class notebook and a post on the discussion board. The examples here use group 7's image, *The Bedroom* by Vincent van Gogh, as in the slides. Local measurements come from the machine that built these notes, which has 32 logical processors and no GPU; GPU measurements come from the class's posts. No recording was used: boxes labelled *Class posts* summarize the discussion board.

Notebooks, slides and reference pages.

Each notebook opens in Google Colab. Choose *Save a copy in Drive* in the *File* menu before you change anything, to save your work.

- The class notebook (https://colab.research.google.com/github/ikoutis/cs301-labs/blob/main/unit-2/notebooks/meeting-11-class.ipynb)
  the two Colab blocks of the meeting, one for each act
- Act 1: A Layer of Neurons Is a Matrix (https://colab.research.google.com/github/ikoutis/cs301-labs/blob/main/unit-2/notebooks/meeting-11-act-1.ipynb)
  the full notebook, Steps 1 to 8
- Act 2: The Same Layer in PyTorch, and on a GPU (https://colab.research.google.com/github/ikoutis/cs301-labs/blob/main/unit-2/notebooks/meeting-11-act-2.ipynb)
  the full notebook, Steps 9 to 15; it asks for a GPU in its first step
- CPU and GPU (https://ikoutis.github.io/cs301-labs/unit-2/pages/cpu-and-gpu.html) the two processors, what is inside a core, how one product is divided, and the two memories

The four slide decks of the meeting, each available as a web page with explanatory notes and as a PDF:

- Meeting 10, Revisited (https://ikoutis.github.io/cs301-labs/unit-2/slides/m11-act-0.html) Act 0 (PDF)
- A Layer of Neurons Is a Matrix (https://ikoutis.github.io/cs301-labs/unit-2/slides/m11-act-1.html) Act 1 (PDF)
- The Same Layer in PyTorch, and on a GPU (https://ikoutis.github.io/cs301-labs/unit-2/slides/m11-act-2.html)
  Act 2 (PDF)
- CPU and GPU (https://ikoutis.github.io/cs301-labs/unit-2/slides/m11-cpu-gpu.html) the companion deck of Act 2 (PDF)

The full notebooks are the reference for Review Quiz 11, whose questions name their steps. The cost of the product, its count of operations and the notation O( ), is developed in the notes of meeting 12.

The main points

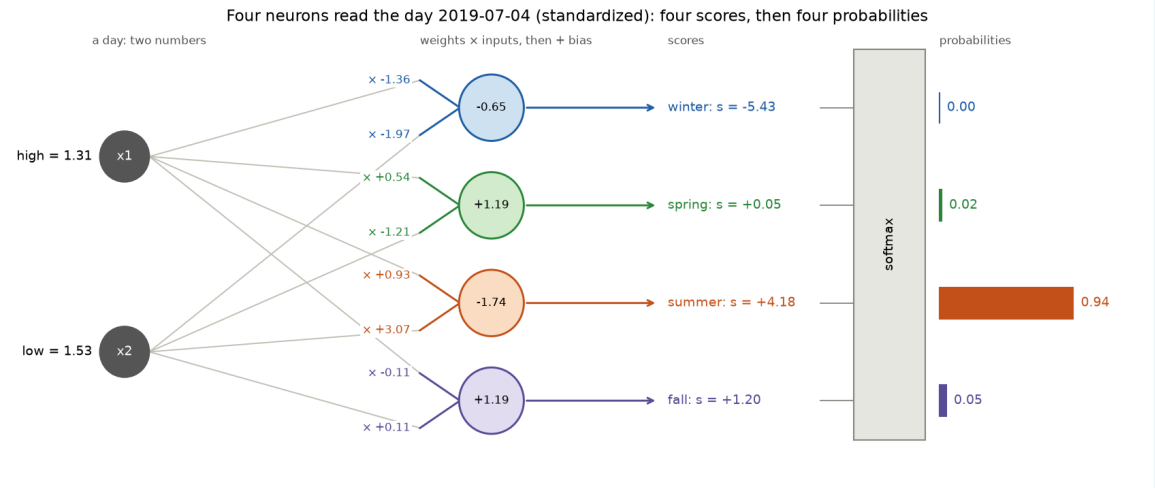
The following ten ideas summarize the meeting. Each links to its explanation and code.

1 A neuron converts a point into a score

A neuron has one **weight** for each coordinate of a point and one more number, its **bias**. For a point x with m coordinates, its score is

$$s = w_1 x_1 + w_2 x_2 + \ \ldots \ + w_m x_m + b = w^T x + b\,.$$

The neuron **fires** when $s \geq 0$; the number $\theta = -b$ is its **threshold**. With w and x as columns, w^T is a row, and a row times a column is one number. In NumPy, `w` and `x` are arrays of shape `(m,)`, and `w @ x + b` is their dot product plus the bias. The weights of the season neurons below were found by **training**: a program adjusted them in small steps to name the right season for as many days of 2006 to 2017 as it could.

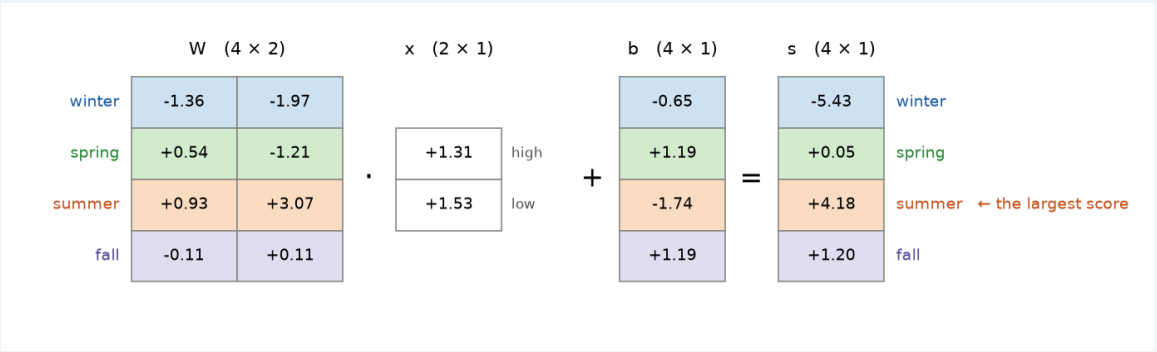


Four neurons read one day, July 4, 2019. The scores pass through a softmax and become probabilities.

Sections 1.2 and 1.6.

2 A layer is a matrix with one neuron in each row

A **layer** is a set of neurons that read the same point. A layer of K neurons is stored as a matrix W with K rows and m columns, **row j holding the weights of neuron j**, and a vector b of K biases. The K scores of one point are $s = Wx + b$: W is $K \times m$ and x is $m \times 1$, so Wx and b are both $K \times 1$. When each neuron stands for one possible answer, the layer's answer is the neuron with the largest score, the **argmax** of s.



Four neurons and one day: s = Wx + b, with the summer neuron's score the largest.

Section 1.3.

3 A layer applied to a batch is one matrix product

Store n points as the rows of X, an $n \times m$ matrix. The scores of all K neurons on all n points form an $n \times K$ matrix,

$$S = XW^T + 1\,b^T,$$

written `X @ W.T + b` in NumPy. Row i of S belongs to point i and column j to neuron j. W is transposed because both a point and a neuron are stored as rows, and the product needs each neuron's weights as a column. The term $1\,b^T$ repeats the row of biases on every row; NumPy does this by **broadcasting**. The transpose moves by the rule $(AB)^T = B^T A^T$; the order of the factors of a matrix product cannot be exchanged.

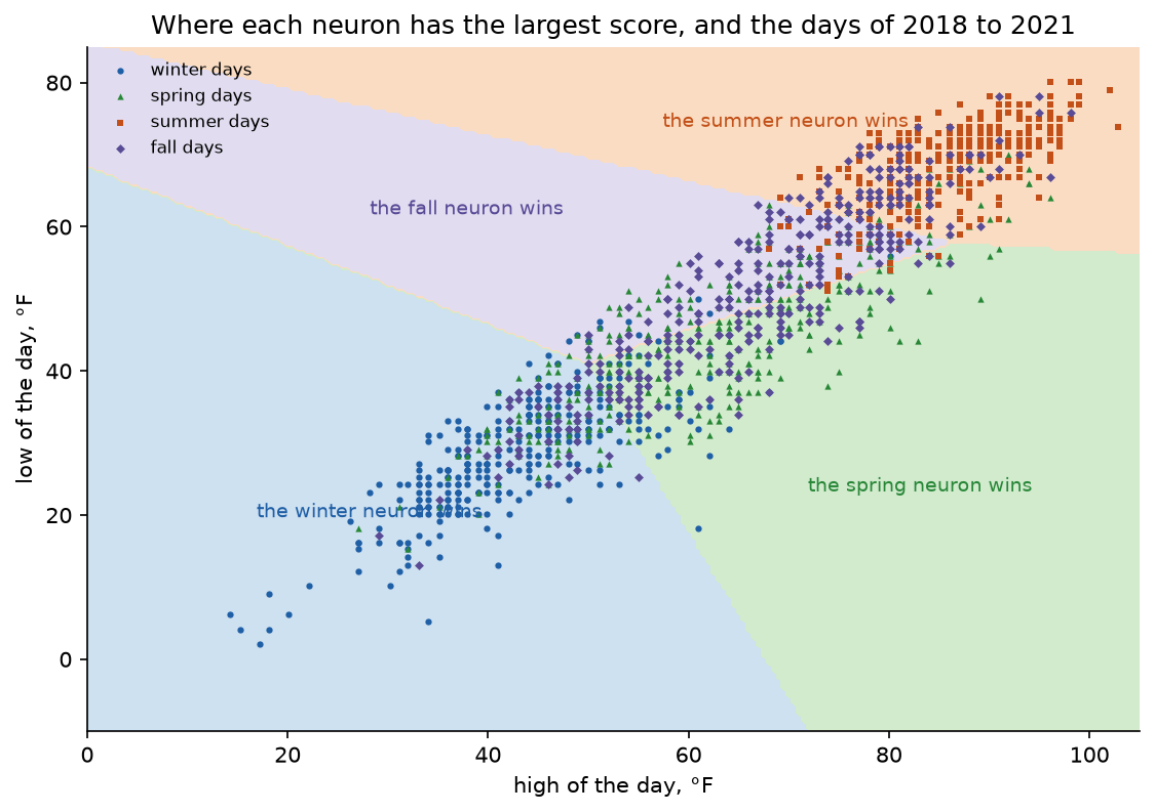
	one point `x`, shape `(m,)`	n points `X`, shape `(n, m)`
one neuron: `w` of shape `(m,)`, `b` a number	`w @ x + b` is a number	`X @ w + b` has shape `(n,)`
K neurons: `W` of shape `(K, m)`, `b` of shape `(K,)`	`W @ x + b` has shape `(K,)`	`X @ W.T + b` has shape `(n, K)`

The shapes do not always expose a missing transpose. When $K = m$, `X @ W + b` runs without an error and computes the wrong scores.

Sections 1.5 and 1.8.

4 The largest score divides the space of points into regions with straight boundaries

The four season neurons named the right season on 59.3% of the 1,457 days of 2018 to 2021, which training had not used. Neurons j and k have equal scores where $(w_j - w_k)^T z + (b_j - b_k) = 0$, a linear equation in the standardized point z: in two coordinates, a line. Standardization shifts and scales each coordinate, so the line is also straight in degrees. The region where one neuron's score is largest is therefore bounded by segments of such lines. Spring and fall days overlap in the plane of highs and lows, so no rule that reads only these two numbers can separate them.



Where each neuron has the largest score, and the days of 2018 to 2021 in the colours of their true seasons.

Section 1.5.

5 The sigmoid acts on each entry; the softmax acts on each row

The **sigmoid** $\sigma(s) = 1 / (1 + e^{-s})$ maps each score separately to a number between 0 and 1, and its result has the shape of its input. The **softmax** maps the K scores of one point to K positive numbers that add to 1,

$$p_k = \frac{e^{s_k}}{e^{s_1} + e^{s_2} + \ \ldots \ + e^{s_K}},$$

so each probability depends on every score of its row. On a matrix of scores the softmax treats each row separately. Subtracting the row's largest entry before the exponential leaves the result unchanged and keeps every exponential at most 1, which avoids overflow. Dividing the scores by a **temperature** $T > 0$ before the softmax multiplies the differences between scores by $1 / T$. For the day of idea 1:

OUTPUT

```
the four scores:         winter -5.43, spring +0.05, summer +4.18, fall +1.20
their probabilities:     winter 0.000, spring 0.015, summer 0.937, fall 0.048   (they
add to 1.000)
sigmoid of each score:   winter 0.004, spring 0.513, summer 0.985, fall 0.769   (each
entry by itself)
```

Sections 1.4, 1.9 and 1.10.

6 A layer can name the nearest reference colour

Give neuron j the weights c_j, a reference colour, and the bias $-\frac{1}{2}\|c_j\|^2$, so that its score is $s_j = c_j^T x - \frac{1}{2}\|c_j\|^2$. Expanding the squared distance gives

$$\|x - c_j\|^2 = \|x\|^2 - 2\left(c_j^T x - \tfrac{1}{2}\|c_j\|^2\right) = \|x\|^2 - 2s_j\,.$$

The term $\|x\|^2$ is the same for every neuron, so the neuron with the largest score has the colour nearest to the pixel. The notebook finds five reference colours of the image with the method k-means, which is outside this meeting. Each row of W is a colour, and can be drawn in that colour.

W (5 × 3): row j, in its own colour

	red	green	blue
neuron 0	0.32	0.36	0.31
neuron 1	0.63	0.46	0.10
neuron 2	0.53	0.53	0.45
neuron 3	0.48	0.57	0.66
neuron 4	0.67	0.65	0.55

The weights of the colour layer: row j of W is reference colour j.

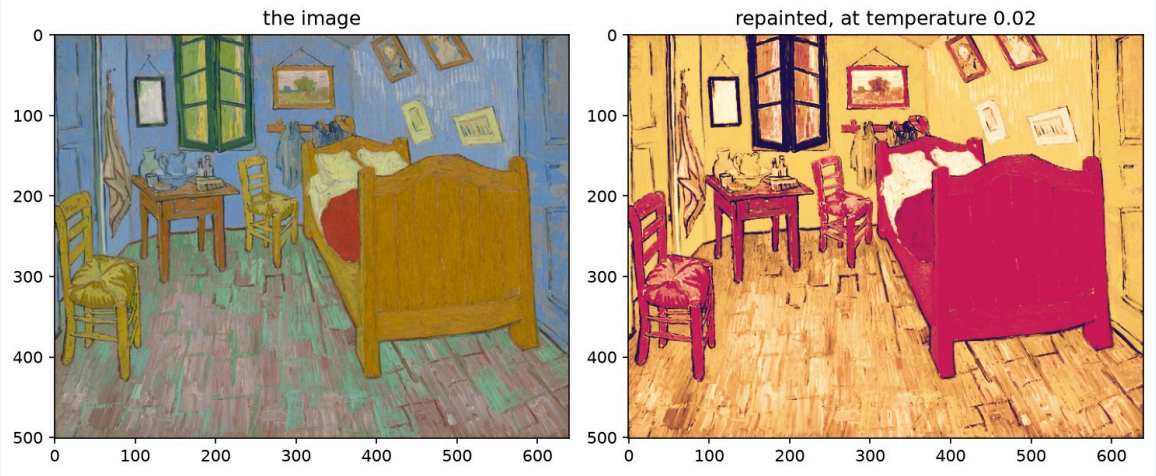
Section 1.7.

7 Two layers with a function between them form a small neural network

Give each neuron a paint, a colour stored as a row of a $K \times 3$ matrix `PAINT`. Then

```
Prob = softmax_rows(layer(X, W, b) / T)    # n x K probabilities
new = Prob @ PAINT                         # n x 3 colours
```

gives every pixel a mixture of the paints, each weighted by its neuron's probability. The second product has the same form as the first: n points with K coordinates, read by three neurons, one for each channel of the new colour. Two layers with a function between them are a small **neural network**, here applied to all the pixels at once. As $T \to 0$, the probabilities of a pixel approach 1 for the neuron with the largest score, when that neuron is unique, and the pixel takes its paint. As T grows, the probabilities approach $1 / K$, and every pixel approaches the average of the paints.



The Bedroom, repainted at temperature 0.02 with the notebook's five paints.

Section 1.10.

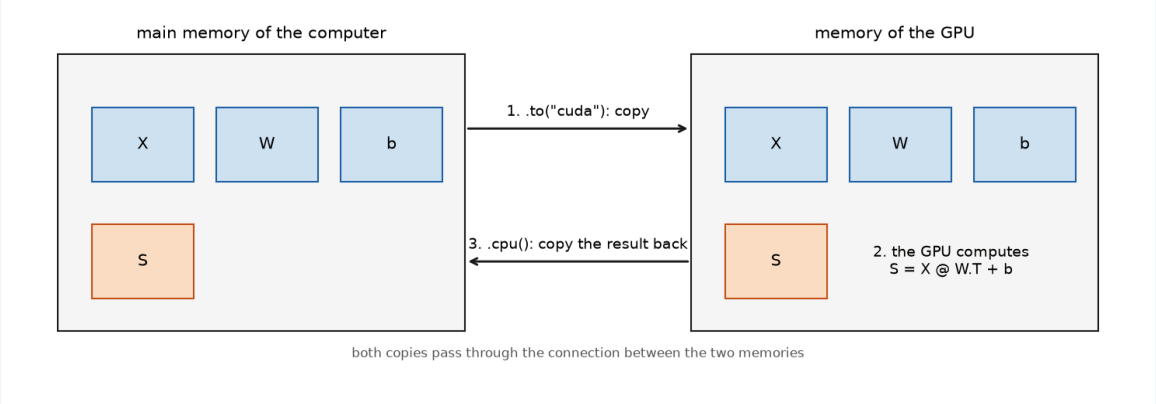
8 PyTorch runs the same line on tensors

PyTorch's arrays are called **tensors**. In this course a tensor is a data structure: an array of numbers with some number of dimensions. `X @ W.T + b`, the transpose `.T` and broadcasting mean what they mean in NumPy. Three differences matter here. PyTorch's default decimal type is `float32`, with 4 bytes and about 7 significant decimal digits, against NumPy's `float64`, with 8 bytes and about 16. PyTorch does not multiply tensors of different types. And reductions along a dimension take `dim` and `keepdim`; `S.max(dim=1)` returns the largest values together with their positions. PyTorch's own layer, `torch.nn.Linear(m, K)`, stores its weights with shape `(K, m)`, one neuron in each row, and computes $y = xA^T + b$.

Sections 2.1 to 2.4.

9 A tensor lives on a device

Every tensor is stored on a **device**: in the main memory, where the CPU computes, or in the memory of a GPU. `.to("cuda")` returns a copy in the GPU's memory, and `.cpu()` copies a result back. An operation needs all of its tensors on one device, and the processor of that device carries it out. PyTorch hands work to a GPU and continues before the work is done, so a timing must call `torch.cuda.synchronize()` before it reads the clock.

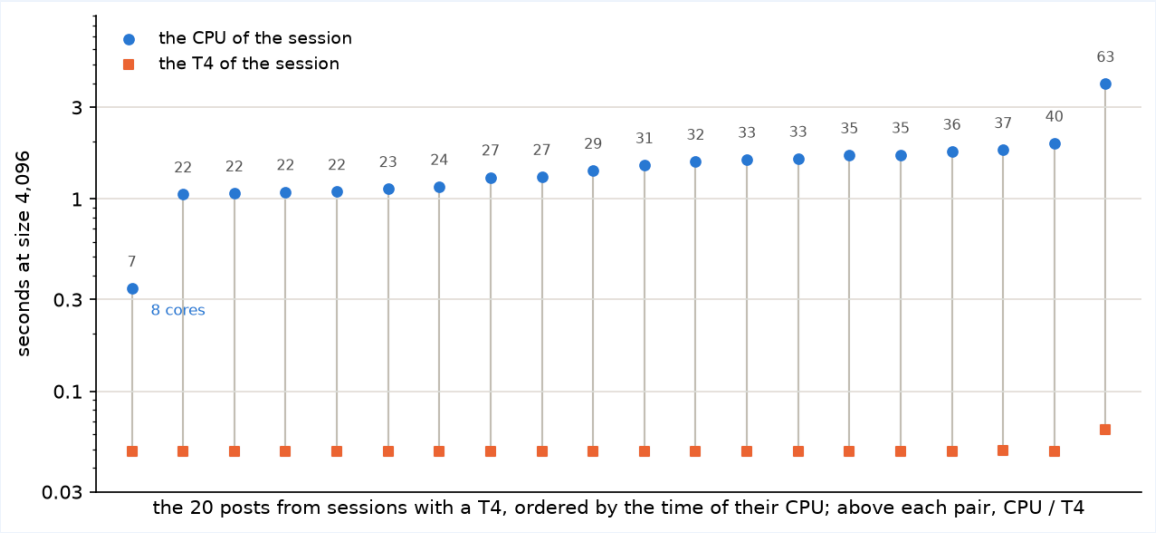


Two memories: the copies to the GPU and back pass through the connection between them.

Sections 2.5 and 2.6.

10 On large products the T4 was tens of times faster than Colab's CPU

Each entry of S is a sum of products that depends on no other entry, so the work can be divided among any number of cores. A CPU has a few cores; the T4 GPU that Colab lends has 2,560. In the class's posts, the product `X @ W.T + b` with $n = m = K = 4{,}096$ took the T4 between 0.0486 and 0.0490 seconds in 19 of 20 sessions, and Colab's two-core CPU between 1.06 and 1.95 seconds in all but one session. The median factor was 31 at size 4,096 and 16 at size 512.



The class's measurements at size 4,096: the CPU and the T4 of each session, with the factor between them.

Section 2.7.

Working with the notebooks, and with an AI assistant.

Run each notebook from the top with Shift+Enter. Complete each **Your turn** exercise, then run its check; a check states what to fix when something is wrong. Two steps ask an AI assistant to write code: Step 6 of Act 1, the function `softmax_rows`, and Step 11 of Act 2, its translation into PyTorch. Put your request in the cell marked **Our prompt**, stating the function's name, its argument, what it returns, and what it may not use. In class the slides showed both functions already written and the class notebook supplied the first, so the one line that the groups wrote in the blocks was the product of Step 4.

Act 0. Meeting 10, revisited

The meeting opened with six ideas of meeting 10 that it builds on. Each is developed, with its code, in the notes of meeting 10.

- **Vectorization.** The same arithmetic, written as one operation on a whole array, runs inside NumPy's compiled code; a Python loop over the entries of an array is the slowest version. Meeting 10, idea 2.
- **Broadcasting.** `X - mu` subtracts m numbers from every row of an $n \times m$ array without storing n copies. Shapes are compared from the right, and a missing dimension, or one of length 1, is repeated. In this meeting broadcasting adds the biases of a layer. Idea 4.
- **Slicing.** `X[0]` is a row, `X[:, 0]` a column, `image[:, :, 0]` a channel. In this meeting, `S[:, j]` holds the scores of neuron j. Idea 5.
- **Axes.** The axis named in `X.mean(axis=0)` is the dimension that disappears. In this meeting, `S.max(axis=1)` and `S.argmax(axis=1)` act along each row. Idea 6.
- **Word vectors.** `U @ v` compares one word with 50,000 words in one product. It is one neuron, whose weights are the vector `v`, applied to 50,000 points. Idea 7.
- **Batches.** N points of the same shape are stored in one array whose first dimension counts the points. Idea 8.

Meeting 10 stored data as arrays. This meeting stores rules that read the data, neurons, in the same way.

Act 1. A Layer of Neurons Is a Matrix

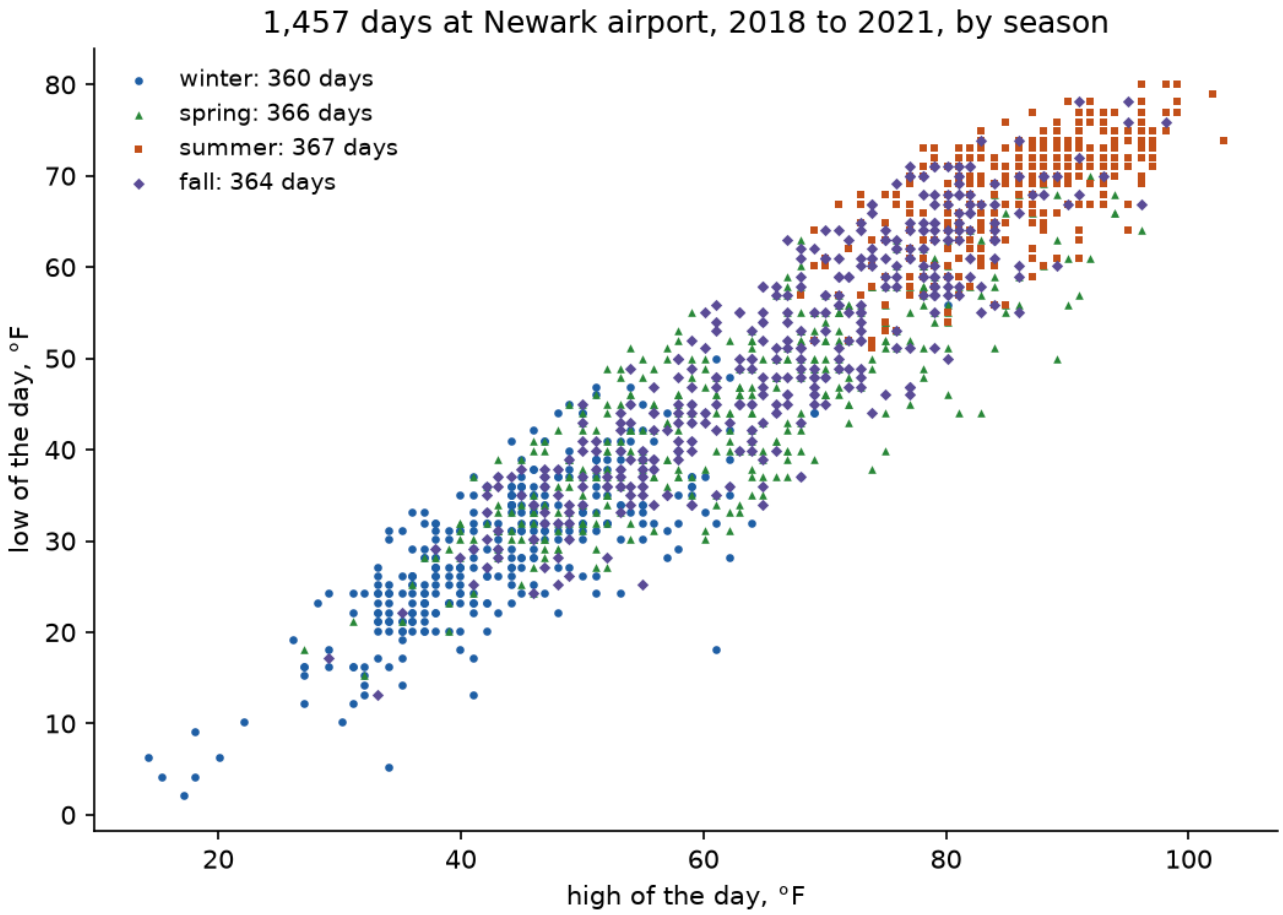
Open the notebook of Act 1 (https://colab.research.google.com/github/ikoutis/cs301-labs/blob/main/unit-2/notebooks/meeting-11-act-1.ipynb)

The slides developed the act on days of weather first and then repeated each idea on the pixels of the group's image. The full notebook takes the pixels first and puts the days in its last step. These notes follow the meeting's order; each section names the step of the notebook where its code appears.

1.1 The data: days at Newark airport Step 8

The dataset holds the daily weather at Newark airport from 2006 to 2024. One **point** is one day, with $m = 2$ coordinates: its high and its low temperature in °F. The days of 2018 to 2021 form the rows of one matrix with $n = 1{,}457$ rows. Each column is standardized, as in meeting 10, with means and standard deviations fixed in the notebook. The **label** of a day is its season, determined by its

month: winter is December to February, spring March to May, summer June to August, and fall September to November.



The days of 2018 to 2021 by their high and low temperatures, coloured by season.

Code: the days, their labels, and the trained layer of four neurons — cells `seasons`, `seasons-data`

```
weather = pd.read_csv(SITE + "weather/newark-weather.csv")
# the days with no missing measurement
weather = weather.dropna(subset=["tmax_f", "tmin_f", "humidity_pct", "pressure_hpa"])
days = weather[weather["date"].str[:4].isin(["2018", "2019", "2020", "2021"])]
month = days["date"].str[5:7].astype(int).to_numpy()
season = (month % 12) // 3              # 0 winter (December to February), 1 spring, 2 summer, 3 fall

X_days = days[["tmax_f", "tmin_f"]].to_numpy()              # the high and the low of each day
mu, sd = np.array([64.7888, 47.9889]), np.array([18.5731, 16.9396])
Z_days = (X_days - mu) / sd                                 # standardized, as in meeting 10

W_seasons = np.array([[-1.35750, -1.96665],      # the neuron of winter
                      [ 0.54402, -1.21157],      # the neuron of spring
                      [ 0.92758,  3.07303],      # the neuron of summer
                      [-0.11410,  0.10518]])     # the neuron of fall
b_seasons = np.array([-0.64726, 1.19295, -1.73890, 1.19321])
```

```
S_days = Z_days @ W_seasons.T + b_seasons        # the line of Step 4
print("shape of Z_days:", Z_days.shape, "  shape of W_seasons:", W_seasons.shape)
print("shape of S_days:", S_days.shape)
right = (S_days.argmax(axis=1) == season).mean()
print(f"The neuron with the largest score names the right season on {right:.1%} of the {len(days):,}
days.")
```

OUTPUT

```
shape of Z_days: (1457, 2)   shape of W_seasons: (4, 2)
shape of S_days: (1457, 4)
The neuron with the largest score names the right season on 59.3% of the 1,457 days.
```

OUTPUT

```
The daily weather at Newark airport, 6,865 days from 2006 to 2024: the high and the low of each
day, in °F.
The days of 2018 to 2021 are n = 1,457 points with m = 2 coordinates each. The first four:
      date  tmax_f  tmin_f
2018-01-01    20.1     6.3
2018-01-02    27.1    12.2
2018-01-03    30.2    10.2
2018-01-04    29.1    18.1
The label of a day is its season, from its month: winter is December to February, then spring,
summer, fall.
```

1.2 One neuron, one day: the score by hand

The winter neuron has two weights and a bias. On July 4, 2019, with a high of 89 °F and a low of 74 °F, the standardized day is $z = (1.31, 1.53)$. The score is two products, their sum, and the bias:

OUTPUT

```
Day 2019-07-04: high 89 °F, low 74 °F. Standardized: z = (1.31, 1.53).
The winter neuron: w = (-1.36, -1.97), b = -0.65.
Its score: -1.36 x 1.31 + (-1.97) x 1.53 + (-0.65) = -5.43
shape of w: (2,), shape of z: (2,); w @ z + b is one number.
```

The score is large and negative: the winter neuron does not fire on this day. As matrices, w and z are 2×1 columns, w^T is 1×2, and $w^T z$ is 1×1, one number. Without the transpose, the product of two 2×1 matrices is not defined. In NumPy both arrays have one dimension, shape `(2,)`, and `@` between them is their dot product.

1.3 Four neurons, one day: the weights are a matrix

Four neurons, one for each season, read the same day. Their weights are the four rows of a 4×2 matrix W, and their biases a vector b of four numbers. One line computes the four scores:

OUTPUT

```
W @ z + b, shape (4,): winter -5.43, spring +0.05, summer +4.18, fall +1.20
shape of W: (4, 2), shape of b: (4,)
The largest score is that of the summer neuron, at position 2: the argmax.
```

Here the sum $Wz + b$ adds two columns of the same shape, 4×1, so nothing has to be repeated. The largest score belongs to the summer neuron, at position 2 counting from 0: the layer's answer for this day.

Code: one neuron and four neurons on one day — cell `seasons-one-day`

```
DAY = "2019-07-04"
if not (days["date"] == DAY).any():
    DAY = days["date"][days["date"].str.startswith("2019-07")].iloc[0]
k_day = int(np.flatnonzero(days["date"].to_numpy() == DAY)[0])
x_day = X_days[k_day]                       # the high and the low of the day, in °F
z_day = Z_days[k_day]                       # the same day, standardized
w_winter, b_winter = W_seasons[0], b_seasons[0]
s_winter = w_winter @ z_day + b_winter
print(f"Day {DAY}: high {x_day[0]:.0f} °F, low {x_day[1]:.0f} °F. Standardized: z = ({z_day[0]:.2f},
{z_day[1]:.2f}).")
print(f"The winter neuron: w = ({w_winter[0]:.2f}, {w_winter[1]:.2f}), b = {b_winter:.2f}.")
print(f"Its score: {w_winter[0]:.2f} x {z_day[0]:.2f} + ({w_winter[1]:.2f}) x {z_day[1]:.2f} +
({b_winter:.2f}) = {s_winter:.2f}")
print(f"shape of w: {w_winter.shape}, shape of z: {z_day.shape}; w @ z + b is one number.")
s_day = W_seasons @ z_day + b_seasons       # the four scores of the day: one line
print(f"W @ z + b, shape {s_day.shape}: " + ", ".join(f"{name} {value:+.2f}" for name, value in
zip(SEASONS, s_day)))
print(f"shape of W: {W_seasons.shape}, shape of b: {b_seasons.shape}")
print(f"The largest score is that of the {SEASONS[int(s_day.argmax())]} neuron, at position
{int(s_day.argmax())}: the argmax.")
```

1.4 From scores to probabilities: the softmax and the sigmoid Step 6

The softmax turns the four scores of the day into four probabilities: summer 0.937, fall 0.048, spring 0.015 and winter 0.000. The sigmoid of each score, by contrast, is computed separately, and the four results do not add to 1: the sigmoid of the spring score, 0.05, is 0.513, close to one half.

The function `softmax_rows` below was written by an AI assistant to the requirements of Step 6: a NumPy array with n rows and K columns in, an array of the same shape out, no loop, and no library softmax. `S.max(axis=1, keepdims=True)` computes the largest entry of each row and keeps the result as an $n \times 1$ array, so that broadcasting subtracts it from every entry of its row. Adding the same number c to all the scores of a row multiplies every exponential by e^c, which cancels in the division; after the largest entry is subtracted, no exponential exceeds 1. The check includes a row of scores near 1,000, whose exponentials would overflow without the subtraction.

Code: Step 6, the softmax of every row, and its check — cells `paste-softmax`, `run-check-softmax`, `seasons-softmax`

```
def softmax_rows(S):
    shifted = S - S.max(axis=1, keepdims=True)      # the largest entry of every row becomes 0
    E = np.exp(shifted)
    return E / E.sum(axis=1, keepdims=True)
```

```
assert "softmax_rows" in globals(), "Paste the function softmax_rows into the cell above and run that
cell."
assert not has_loop(softmax_rows), "This function must use operations on whole arrays: no loop."
assert not mentions(softmax_rows, ("scipy", "torch", "sklearn")), (
    "This function must compute the softmax itself, with NumPy only: no softmax function from a library.")

test = np.array([[-1.0, 0.0, 2.0, 1.0],
                 [0.0, 0.0, 0.0, 0.0],
                 [1000.0, 1003.0, 1001.0, 1002.0]])
expected = np.array([[0.0321, 0.0871, 0.6439, 0.2369],
                     [0.25, 0.25, 0.25, 0.25],
                     [0.0321, 0.6439, 0.0871, 0.2369]])
try:
    with np.errstate(all="ignore"):
        result = np.asarray(softmax_rows(test.copy()), dtype=float)
except ValueError:
    raise AssertionError(
        "The shapes inside softmax_rows do not fit. The largest entry and the sum of every row must keep "
        "the shape (n, 1), so that they can be combined with the n rows: use keepdims=True.") from None
assert result.shape == test.shape, f"softmax_rows returned an array of shape {result.shape}; it must be
{test.shape}."
assert not np.isnan(result).any(), (
    "softmax_rows returns nan (not a number) for the row of very large scores. "
    "Subtract the largest entry of each row before the exponential.")
assert np.allclose(result.sum(axis=1), 1), (
    "The rows of the result do not add to 1. The largest entry and the sum must be taken along each row:
axis=1.")
assert np.allclose(result, expected, atol=2e-4), (
    f"The result is not the softmax of the rows. For the scores -1, 0, 2, 1 the function gives
{np.round(result[0], 2)}; "
    "the right values are 0.03, 0.09, 0.64, 0.24.")
print("Correct: every row of the result adds to 1, also for very large scores.")
print("The scores -1, 0, 2, 1 give", np.round(result[0], 2))
```

OUTPUT

```
Correct: every row of the result adds to 1, also for very large scores.
The scores -1, 0, 2, 1 give [0.03 0.09 0.64 0.24]
```

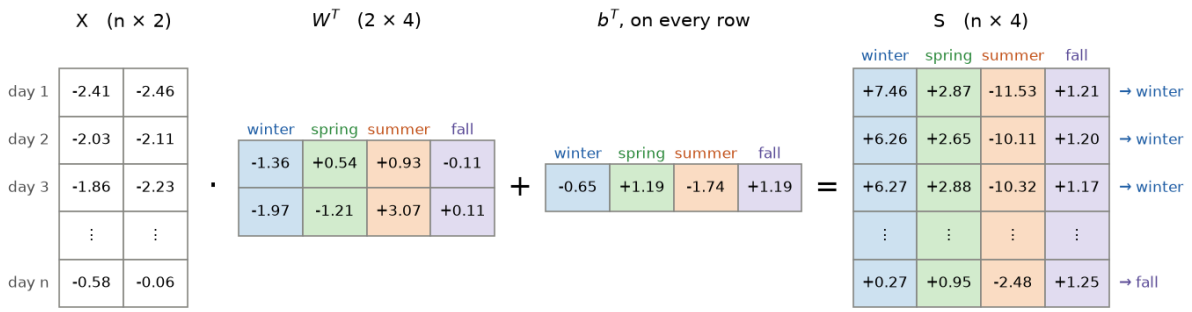
```
p_day = softmax_rows(s_day.reshape(1, 4))[0]        # the softmax of one row of four scores
print("the four scores:        " + ", ".join(f"{name} {value:+.2f}" for name, value in zip(SEASONS,
s_day)))
print("their probabilities:    " + ", ".join(f"{name} {value:.3f}" for name, value in zip(SEASONS, p_day))
+ f"   (they add to {p_day.sum():.3f})")
print("sigmoid of each score:  " + ", ".join(f"{name} {value:.3f}" for name, value in zip(SEASONS,
sigmoid(s_day))) + "   (each entry by itself)")
```

OUTPUT

```
the four scores:        winter -5.43, spring +0.05, summer +4.18, fall +1.20
their probabilities:    winter 0.000, spring 0.015, summer 0.937, fall 0.048   (they add to
1.000)
sigmoid of each score:  winter 0.004, spring 0.513, summer 0.985, fall 0.769   (each entry by
itself)
```

1.5 Four neurons, many days: one product

For all the days at once, `Z_days @ W_seasons.T + b_seasons` computes an $n \times 4$ matrix of scores: one row for each day and one column for each neuron. In mathematical notation, $S = XW^T + 1\,b^T$, where X is $n \times 2$ and W^T is 2×4. Row i of S is the column $Wz_i + b$ of section 1.3, written as a row. The row of biases b^T is added to every row of XW^T: the column of ones 1 writes this out in mathematics, and broadcasting does it in NumPy.



The scores of all the days in one product. The columns of the transposed weights and of S belong to the four neurons.

The largest score of each row names a season, and it is right on 59.3% of the days. The picture in idea 4 colours the plane by the neuron with the largest score: its boundaries are straight, and spring and fall overlap. The table of the four cases in idea 3 summarizes the shapes of this act.

Code: the regions of the four neurons — cell `seasons-many`

```
print("shape of Z_days:", Z_days.shape, "  shape of W_seasons.T:", W_seasons.T.shape, "  shape of
S_days:", S_days.shape)
named = S_days.argmax(axis=1)               # for every day, the neuron with the largest score
print(f"The largest score names the right season on {(named == season).mean():.1%} of the {len(days):,}
days.")
highs, lows = np.linspace(0, 105, 211), np.linspace(-10, 85, 191)
HH, LL = np.meshgrid(highs, lows)
grid_points = np.column_stack([HH.ravel(), LL.ravel()])
region = (((grid_points - mu) / sd) @ W_seasons.T + b_seasons).argmax(axis=1).reshape(HH.shape)
fig, ax = plt.subplots(figsize=(7.6, 5.4))
ax.contourf(HH, LL, region, levels=[-0.5, 0.5, 1.5, 2.5, 3.5], colors=fills)
for k in range(4):
    sel = season == k
    ax.scatter(X_days[sel, 0], X_days[sel, 1], s=9, marker=markers[k], color=dark[k], linewidths=0,
label=f"{SEASONS[k]} days")
    inside = region == k
    if inside.mean() > 0.04:
        ax.text(HH[inside].mean(), LL[inside].mean(), f"the {SEASONS[k]} neuron wins", ha="center",
va="center",
                fontsize=9, color=dark[k])
ax.set_xlabel("high of the day, °F")
ax.set_ylabel("low of the day, °F")
ax.set_title("Where each neuron has the largest score, and the days of 2018 to 2021")
```

```
ax.legend(loc="upper left", frameon=False, fontsize=8)
ax.spines[["top", "right"]].set_visible(False)
fig.tight_layout()
plt.show()
```

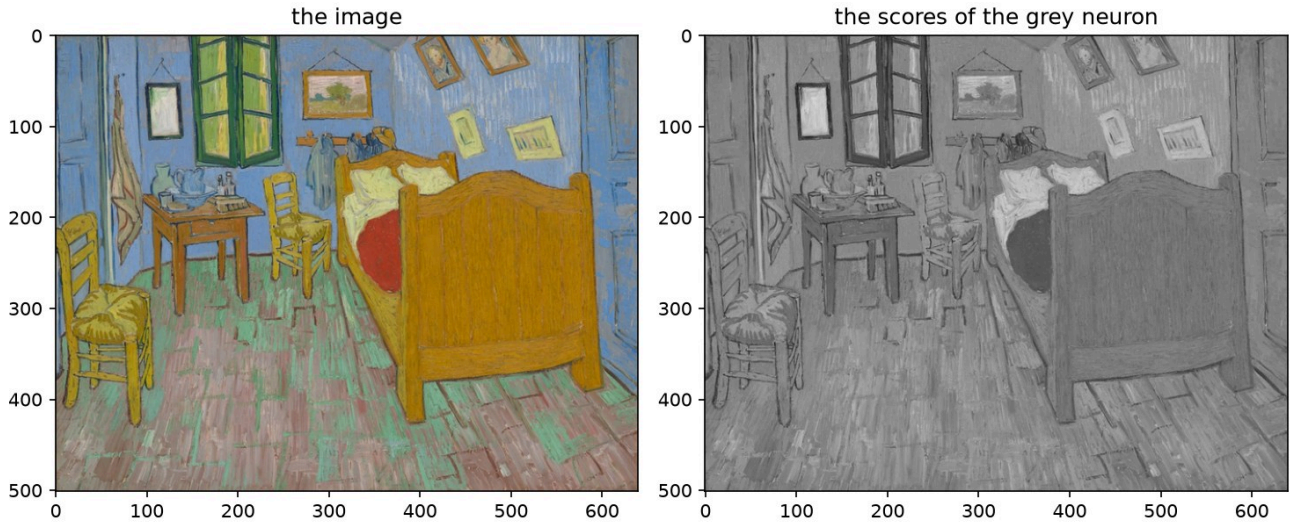
OUTPUT

```
shape of Z_days: (1457, 2)   shape of W_seasons.T: (2, 4)   shape of S_days: (1457, 4)
The largest score names the right season on 59.3% of the 1,457 days.
```

1.6 A second example: a neuron that reads a pixel Steps 1 and 2

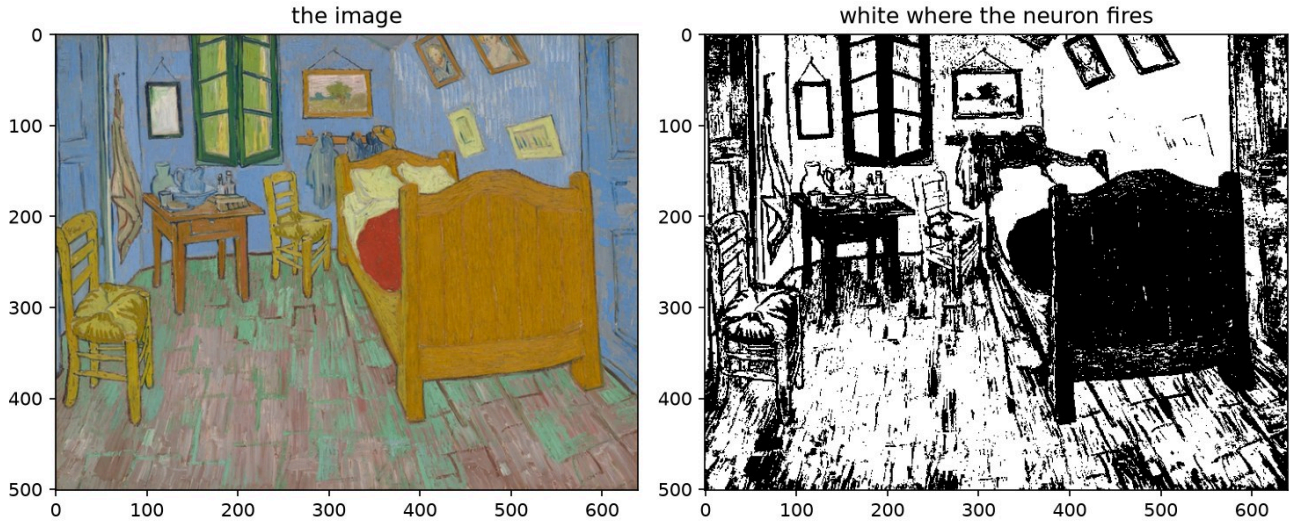
In the second example a point is one **pixel** of the group's image, with $m = 3$ coordinates: its red, green and blue values, each from 0 to 1. What counts as a point depends on the question asked, and these questions are about colours. `image.reshape(-1, 3)` writes the pixels as the rows of `X`. For *The Bedroom*, n = 320,640 points.

The grey rule of meeting 10 is a neuron, with weights 0.299, 0.587 and 0.114 and bias 0: its score is the brightness of the pixel. On the middle pixel, (0.56, 0.39, 0.10), its score is 0.411. The hand computation of Step 1 uses these rounded values and may differ in the third decimal. For all the pixels, `X @ w_grey + b_grey` gives one score for each point. Reshaped to the image's height and width, the scores are the grey image of meeting 10: the largest difference from the stored grey image is 0.0020.



The scores of the grey neuron, drawn at the places of their pixels: the grey image.

A brightness is never negative, so with bias 0 the grey neuron fires on every pixel. With bias -0.518, minus the mean brightness, it fires on the pixels brighter than the average: 53.1% of them. The exercise of Step 2 asks for a neuron of the group's own. The notebook's starting values, weights (1, −0.5, −0.5) and bias −0.1, fire where the red value exceeds the average of green and blue by at least 0.1: on 36.4% of the pixels here.



White where the grey neuron fires, with its threshold at the mean brightness.

This course, like pandas, NumPy and PyTorch, stores a point as a row, so the scores of one neuron on many points are $Xw + b$. Many books store points as columns. Their data matrix is X^T, and the same scores are written $w^T X^T + b$. Step 2 computes both and checks that they agree.

Code: Steps 1 and 2, one neuron on one pixel and on all of them

cells `image-number`, `one-unit-one-point`, `transpose`, `one-unit-many-points`, `fires`, `own-unit`, `two-conventions`

```
IMAGE = 7        # replace 0 with your group's number, from 1 to 30

assert 1 <= IMAGE <= 30, "Type your group's number in place of the 0."

images = pd.read_csv(IMAGES + "images.csv", index_col="number")
picture_info = images.loc[IMAGE]
image = read_image(picture_info["file"])
height, width = image.shape[0], image.shape[1]
print(f"Image {IMAGE}: {picture_info['title']} ({picture_info['creator']})")
print(f"shape of image: {image.shape}, which is {height} rows and {width} columns of pixels")
show([image], [f"image {IMAGE}"])

x = image[height // 2, width // 2]             # the pixel in the middle of the image
w_grey = np.array([0.299, 0.587, 0.114])       # the weights of the grey neuron
b_grey = 0.0                                   # its bias
print("the pixel   x =", np.round(x, 2), " (red, green, blue)")
print("the weights w =", w_grey, " and the bias b =", b_grey)
```

OUTPUT

```
Image 7: The Bedroom (Vincent van Gogh)
shape of image: (501, 640, 3), which is 501 rows and 640 columns of pixels
the pixel   x = [0.56 0.39 0.1 ]  (red, green, blue)
the weights w = [0.299 0.587 0.114]  and the bias b = 0.0
```

```
s_one = w_grey @ x + b_grey          # the dot product of the weights and the point, plus the bias
print(f"s = {s_one:.3f}")
```

```
print("shape of w_grey:", w_grey.shape, "  shape of x:", x.shape)
```

OUTPUT

```
s = 0.411
shape of w_grey: (3,)   shape of x: (3,)
```

```
print("shape of w_grey.T:", w_grey.T.shape, "- an array with one dimension is its own transpose")

w_column = w_grey.reshape(3, 1)      # the same numbers as a matrix with 3 rows and 1 column
x_column = x.reshape(3, 1)
print("shape of w_column:", w_column.shape, "  shape of w_column.T:", w_column.T.shape)
print("w_column.T @ x_column =", w_column.T @ x_column, "- a matrix with 1 row and 1 column")
try:
    w_column @ x_column
except ValueError:
    print("w_column @ x_column fails: a matrix with 1 column cannot be multiplied by a matrix with 3
rows")
```

OUTPUT

```
shape of w_grey.T: (3,) - an array with one dimension is its own transpose
shape of w_column: (3, 1)   shape of w_column.T: (1, 3)
w_column.T @ x_column = [[0.41066667]] - a matrix with 1 row and 1 column
w_column @ x_column fails: a matrix with 1 column cannot be multiplied by a matrix with 3 rows
```

```
X = image.reshape(-1, 3)                # one row for each pixel
n, m = X.shape
print(f"shape of X: {X.shape}, which is n = {n:,} points with m = {m} coordinates")

s_grey = X @ w_grey + b_grey            # the scores of the grey neuron on all the points
print("shape of X @ w_grey + b_grey:", s_grey.shape)
print("the scores of the first five points:", np.round(s_grey[:5], 3))

grey = read_image(picture_info["grey_file"])          # the grey image of meeting 10
difference = np.abs(s_grey.reshape(height, width) - grey).max()
print(f"largest difference between the scores and the grey image of meeting 10: {difference:.4f}")
show([image, s_grey.reshape(height, width)], ["the image", "the scores of the grey neuron"])
```

OUTPUT

```
shape of X: (320640, 3), which is n = 320,640 points with m = 3 coordinates
shape of X @ w_grey + b_grey: (320640,)
the scores of the first five points: [0.482 0.532 0.5   0.482 0.477]
largest difference between the scores and the grey image of meeting 10: 0.0020
```

```
b_bright = -s_grey.mean()                       # minus the mean brightness of the image
fires = X @ w_grey + b_bright >= 0              # True for the points on which the neuron fires
print(f"bias {b_bright:.3f}: the neuron fires on {fires.mean():.1%} of the {n:,} pixels")
show([image, fires.reshape(height, width).astype(float)], ["the image", "white where the neuron fires"])
```

OUTPUT

```
bias -0.518: the neuron fires on 53.1% of the 320,640 pixels
```

```
w_own = np.array([1.0, -0.5, -0.5])      # the weights for red, green and blue
b_own = -0.1                             # the bias

fires_own = X @ w_own + b_own >= 0
print(f"weights {w_own}, bias {b_own}: the neuron fires on {fires_own.mean():.1%} of the pixels")
show([image, fires_own.reshape(height, width).astype(float)], ["the image", "white where our neuron
fires"])
```

OUTPUT

```
weights [ 1.  -0.5 -0.5], bias -0.1: the neuron fires on 36.4% of the pixels
```

```
by_rows = X @ w_grey + b_bright          # points as rows: X has shape (n, m)
by_columns = w_grey @ X.T + b_bright     # points as columns: X.T has shape (m, n)
print("shape of X:", X.shape, "  shape of X.T:", X.T.shape)
print("The two results are equal:", np.allclose(by_rows, by_columns))
try:
    w_grey @ X
except ValueError:
    print(f"w_grey @ X fails: w_grey has 3 entries and X has {n:,} rows, so the shapes do not fit")
```

OUTPUT

```
shape of X: (320640, 3)   shape of X.T: (3, 320640)
The two results are equal: True
w_grey @ X fails: w_grey has 3 entries and X has 320,640 rows, so the shapes do not fit
```

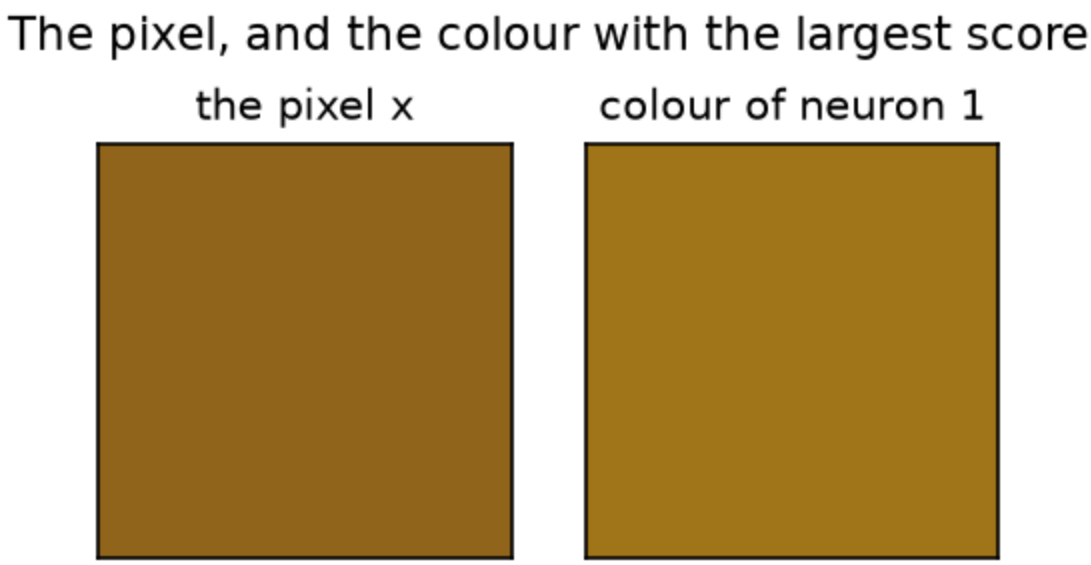
1.7 Five neurons, one for each reference colour Step 3

The layer of this example has $K = 5$ neurons. The setup finds five **reference colours** that occur often in the image, ordered from the darkest to the lightest. Row j of `W` is colour c_j itself, and the bias of neuron j is $-\frac{1}{2}\|c_j\|^2$. By the identity of idea 6, the neuron with the largest score is the one whose colour is nearest to the pixel.

On the middle pixel, `W @ x + b` gives five scores. The largest is that of neuron 1, whose colour is the nearest of the five:

OUTPUT

```
the pixel x = [0.56 0.39 0.1 ]
shape of W @ x + b: (5,)
the scores of the 5 neurons: [0.19  0.237 0.17  0.065 0.104]
The largest score is that of neuron 1, whose colour is [0.63 0.46 0.1 ].
```



The middle pixel, and the reference colour with the largest score.

Code: Step 3, the layer of five neurons, and its scores on one pixel

cells `layer-of-units`, `many-units-one-point`

```
K = 5
C = reference_colours(X, K)              # K rows and 3 columns: the reference colours, darkest first
W = C.copy()                             # the weights of the layer: row j is reference colour j
b = -0.5 * (C ** 2).sum(axis=1)          # the biases of the layer: one number for each neuron

print("shape of W:", W.shape, "  shape of b:", b.shape)
print("W =")
print(np.round(W, 2))
print("b =", np.round(b, 2))
show_colours(C, [f"neuron {j}" for j in range(K)], "The reference colours: row j of W")
```

OUTPUT

```
shape of W: (5, 3)   shape of b: (5,)
W =
[[0.32 0.36 0.31]
 [0.63 0.46 0.1 ]
 [0.53 0.53 0.45]
 [0.48 0.57 0.66]
 [0.67 0.65 0.55]]
b = [-0.16 -0.31 -0.38 -0.49 -0.59]
```

```
s_x = W @ x + b                          # the K scores of the pixel x
print("the pixel x =", np.round(x, 2))
print("shape of W @ x + b:", s_x.shape)
print("the scores of the", K, "neurons:", np.round(s_x, 3))
best = int(s_x.argmax())                 # the position of the largest score
print(f"The largest score is that of neuron {best}, whose colour is {np.round(C[best], 2)}.")
```

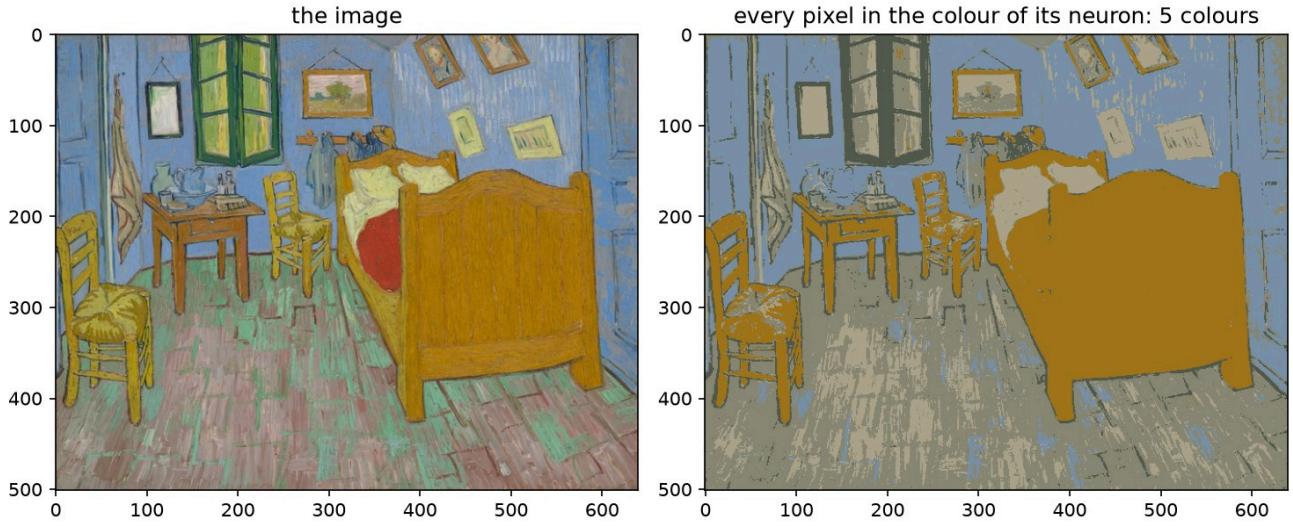
```
show_colours(np.array([x, C[best]]), ["the pixel x", f"colour of neuron {best}"],
             "The pixel, and the colour with the largest score")
```

1.8 Many neurons, many points: three loops or one line Step 4

The scores of every neuron on every point form a matrix `S` with n rows and K columns. `layer_loops` computes them from the definition, with three nested loops: over the points, over the neurons, and over the coordinates. Its innermost line runs $n \times K \times m = 4{,}809{,}600$ times for this image.

The one line that the groups wrote in the Colab block replaces the three loops: `X @ W.T + b`. The check compares it with the loops on the first 2,000 points. Without the transpose, `X @ W` multiplies an $n \times 3$ matrix by a 5×3 matrix, which is not defined, and NumPy raises an error; the check's message asks which array has to be transposed. On the machine that built these notes, the loops took 1.40 seconds and the product 0.0055 seconds, so the loops took about 256 times as long for the same arithmetic. The notes of meeting 12 count these operations and compare the rates of several ways of computing them.

`S.argmax(axis=1)` gives, for every pixel, the neuron with the largest score, and `C[winner]` the reference colour of that neuron. The picture paints every pixel in the nearest of the five colours.



Every pixel in the colour of its neuron: the argmax of each row of S.

Code: Step 4, the three loops, the one line, its check, the measurement, and the argmax cells `layer-loops`, `paste-layer`, `run-check-layer`, `measure-layer`, `argmax`

```
def layer_loops(X, W, b):
    """The scores of the K neurons of a layer on the n points of X, one multiplication at a time."""
    n, m = X.shape
    K = W.shape[0]
    S = np.empty((n, K))
    for i in range(n):                    # every point
```

```
        for j in range(K):                # every neuron
            total = b[j]
            for c in range(m):            # every coordinate
                total += W[j, c] * X[i, c]
            S[i, j] = total
    return S


S_first = layer_loops(X[:2000], W, b)
print("shape of the scores of the first 2,000 points:", S_first.shape)
```

OUTPUT

```
shape of the scores of the first 2,000 points: (2000, 5)
```

```
def layer(X, W, b):
    """The scores of the K neurons of a layer on the n points of X: an array with n rows and K columns."""
    return X @ W.T + b
```

```
try:
    S = layer(X, W, b)
except ValueError:
    raise AssertionError(
        "The shapes do not fit. X has shape (n, 3) and W has shape (K, 3), and the result must have n rows
"
        "and K columns. Which array has to be transposed?") from None
assert S is not ..., "Replace the three dots in the function layer with your line, and run that cell
again."
S = np.asarray(S)
assert S.shape == (n, K), (
    f"layer returned an array of shape {S.shape}. It must be {(n, K)}: one row for each point and one
column for each neuron.")
assert np.allclose(S[:2000], S_first), (
    "The scores differ from those of layer_loops. Check which array is transposed, and that the bias is
added.")
print(f"Correct: layer(X, W, b) has shape {S.shape} and agrees with the three loops.")
```

OUTPUT

```
Correct: layer(X, W, b) has shape (320640, 5) and agrees with the three loops.
```

```
start = time.perf_counter()
S_loops = layer_loops(X, W, b)
t_loops = time.perf_counter() - start

start = time.perf_counter()
for _ in range(20):
    S = layer(X, W, b)
t_product = (time.perf_counter() - start) / 20

print(f"n = {n:,} points, m = {m} coordinates, K = {K} neurons")
print(f"The innermost line of the loops runs n x K x m = {n * K * m:,} times.")
print(f"{'version':16s}{'seconds':>12s}")
print(f"{'three loops':16s}{t_loops:12.4f}")
print(f"{'one product':16s}{t_product:12.4f}")
print(f"The loops take {t_loops / t_product:.0f} times as long as the product.")
```

OUTPUT

```
n = 320,640 points, m = 3 coordinates, K = 5 neurons
The innermost line of the loops runs n x K x m = 4,809,600 times.
version              seconds
three loops           1.4020
one product           0.0055
The loops take 256 times as long as the product.
```

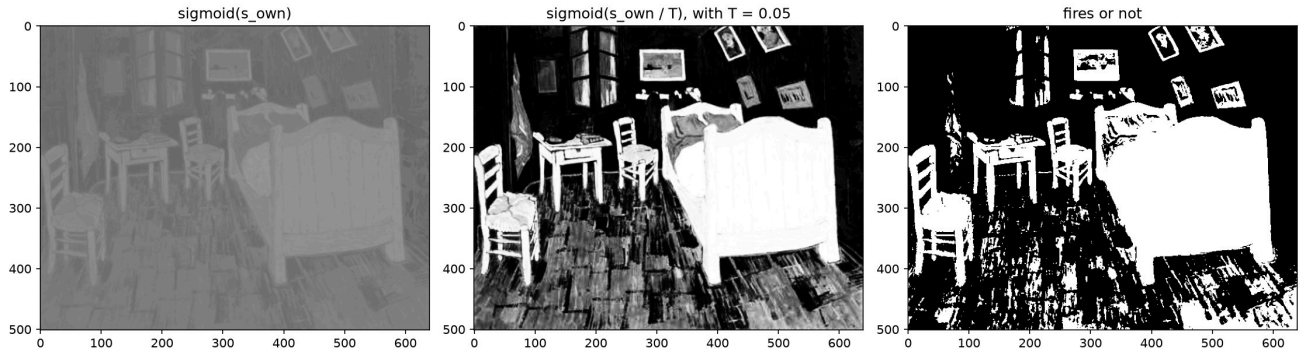
```
winner = S.argmax(axis=1)                # for each point, the neuron with the largest score
print("shape of winner:", winner.shape, "  its first ten entries:", winner[:10])
shares = np.bincount(winner, minlength=K) / n
for j in range(K):
    print(f"neuron {j}, colour {np.round(C[j], 2)}: the largest score on {shares[j]:.1%} of the pixels")
show([image, C[winner].reshape(height, width, 3)],
     ["the image", f"every pixel in the colour of its neuron: {K} colours"])
```

OUTPUT

```
shape of winner: (320640,)   its first ten entries: [2 2 2 2 2 2 2 2 2 2]
neuron 0, colour [0.32 0.36 0.31]: the largest score on 7.8% of the pixels
neuron 1, colour [0.63 0.46 0.1 ]: the largest score on 23.0% of the pixels
neuron 2, colour [0.53 0.53 0.45]: the largest score on 33.1% of the pixels
neuron 3, colour [0.48 0.57 0.66]: the largest score on 24.4% of the pixels
neuron 4, colour [0.67 0.65 0.55]: the largest score on 11.7% of the pixels
```

1.9 The sigmoid on every pixel Step 5

`1 / (1 + np.exp(-s))` applies the sigmoid to every entry of an array of any shape. On the scores of the group's own neuron of Step 2, its values lie between 0.39 and 0.60, because the scores are small numbers near 0. Dividing the scores by a temperature $T = 0.05$ multiplies them by 20, and the values of the sigmoid then spread from 0 to 1. As $T \to 0$, $\sigma(s / T)$ approaches 1 where $s > 0$ and 0 where $s < 0$: the neuron's "fires or not".



sigmoid(s), sigmoid(s / 0.05), and fires-or-not.

Code: Step 5, the sigmoid — cell `sigmoid`

```
def sigmoid(s):
    """The sigmoid of every entry of the array s."""
    return 1 / (1 + np.exp(-s))


s_own = X @ w_own + b_own               # the scores of your own neuron of Step 2 on all the points
T = 0.05                                # the temperature

print("shape of s_own:", s_own.shape, " shape of sigmoid(s_own):", sigmoid(s_own).shape)
print("shape of S:    ", S.shape, " shape of sigmoid(S):    ", sigmoid(S).shape)
print(f"sigmoid(s_own) lies between {sigmoid(s_own).min():.2f} and {sigmoid(s_own).max():.2f}")
print(f"sigmoid(s_own / T) lies between {sigmoid(s_own / T).min():.2f} and {sigmoid(s_own /
T).max():.2f}")
show([sigmoid(s_own).reshape(height, width), sigmoid(s_own / T).reshape(height, width),
      (s_own >= 0).reshape(height, width).astype(float)],
     ["sigmoid(s_own)", f"sigmoid(s_own / T), with T = {T}", "fires or not"])
```

OUTPUT

```
shape of s_own: (320640,)  shape of sigmoid(s_own): (320640,)
shape of S:     (320640, 5)  shape of sigmoid(S):     (320640, 5)
sigmoid(s_own) lies between 0.39 and 0.60
sigmoid(s_own / T) lies between 0.00 and 1.00
```

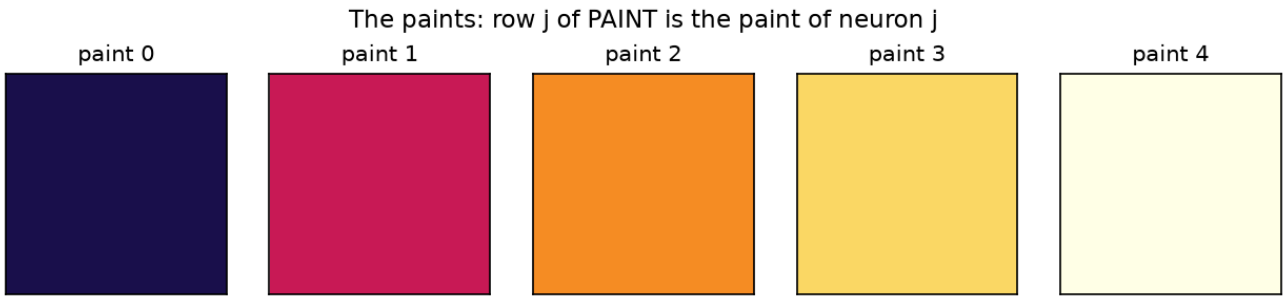
1.10 Probabilities, and the image repainted — Steps 6 and 7

`softmax_rows(S / T)` turns the five scores of every pixel into five probabilities. The scores of one pixel differ by only a few hundredths. At temperature 0.02 those differences are multiplied by 50, and one neuron clearly leads:

OUTPUT

```
the scores of the point in row 0:        [0.295 0.29  0.324 0.279 0.29 ]
its probabilities at temperature 0.02:  [0.137 0.109 0.586 0.062 0.107]
the sum of its probabilities: 1.0
```

Step 7 gives each neuron a paint and computes `Prob @ PAINT`, as in idea 7. A pixel's new colour is the average of the paints weighted by its probabilities. At temperature 1 the probabilities of a pixel are all close to $1 / 5$, so every pixel takes nearly the same mixture and the picture is close to one flat colour. At 0.002 each pixel takes the paint of its nearest reference colour, and the regions have sharp boundaries.



The notebook's five paints, the rows of PAINT.

Class posts.

Twenty-three posts included the line printed under the repainted picture. Nine used temperature 0.002, nine the class notebook's 0.02, and the others 0.01, 0.05 or 0.1. Most posts described the effect of the temperature as above: one flat or muddy colour at temperature 1, sharp regions of single paints at 0.002. A few described a change of hue, such as warmer or more saturated colours. The temperature does not change the paints. It changes only how strongly each pixel's mixture favours the paint of its top neuron.

Code: Steps 6 and 7, the probabilities and the repainting — cells `probabilities`, `repaint`

```
T = 0.02                                # the temperature of the softmax
Prob = softmax_rows(S / T)              # n rows and K columns: the probabilities
print("shape of Prob:", Prob.shape)
print("the scores of the point in row 0:       ", np.round(S[0], 3))
print(f"its probabilities at temperature {T}: ", np.round(Prob[0], 3))
print("the sum of its probabilities:", round(float(Prob[0].sum()), 6))
```

OUTPUT

```
shape of Prob: (320640, 5)
the scores of the point in row 0:         [0.295 0.29  0.324 0.279 0.29 ]
its probabilities at temperature 0.02:  [0.137 0.109 0.586 0.062 0.107]
the sum of its probabilities: 1.0
```

```
PAINT = np.array([[0.10, 0.07, 0.30],      # the paint of neuron 0, the neuron of the darkest reference
colour
                  [0.80, 0.10, 0.35],      # the paint of neuron 1
                  [0.98, 0.55, 0.15],      # the paint of neuron 2
                  [0.99, 0.85, 0.40],      # the paint of neuron 3
                  [1.00, 1.00, 0.92]])     # the paint of neuron 4, the neuron of the lightest reference
colour
T = 0.02                                   # the temperature of the softmax

assert PAINT.shape == (K, 3), f"PAINT must have {K} rows, one for each neuron, and 3 columns."
Prob = softmax_rows(layer(X, W, b) / T)    # n rows and K columns
new = Prob @ PAINT                         # n rows and 3 columns
print("shape of Prob:", Prob.shape, "  shape of PAINT:", PAINT.shape, "  shape of new:", new.shape)
```

```
show_colours(PAINT, [f"paint {j}" for j in range(K)], "The paints: row j of PAINT is the paint of neuron
j")
show([image, new.reshape(height, width, 3)], ["the image", f"repainted, at temperature {T}"])

paints = "; ".join(str(tuple(float(v) for v in np.round(paint, 2))) for paint in PAINT)
print("Line for the discussion board:")
print(f"Image {IMAGE} ({picture_info['title']}), repainted with {K} paints at temperature {T}. "
      f"Paints (red, green, blue): {paints}.")
```

OUTPUT

```
shape of Prob: (320640, 5)   shape of PAINT: (5, 3)   shape of new: (320640, 3)
Line for the discussion board:
Image 7 (The Bedroom), repainted with 5 paints at temperature 0.02. Paints (red, green, blue):
(0.1, 0.07, 0.3); (0.8, 0.1, 0.35); (0.98, 0.55, 0.15); (0.99, 0.85, 0.4); (1.0, 1.0, 0.92).
```

Act 2. The Same Layer in PyTorch, and on a GPU

Open the notebook of Act 2 (https://colab.research.google.com/github/ikoutis/cs301-labs/blob/main/unit-2/notebooks/meeting-11-act-2.ipynb)

PyTorch is the library in which most neural networks are written. The act repeats the layer of Act 1 in PyTorch, moves it to a GPU, and times one large product on both processors. Colab assigns a GPU only to a session that asks for one, and changing the runtime type erases the session. The class notebook and Step 9 therefore ask for a T4 GPU before anything is run; every cell also runs without one.

2.1 From arrays to tensors Step 10

Arrays are named after their number of dimensions, and all of them are **tensors**. In mathematics, tensors of more than two dimensions have a theory of their own; in this course a tensor is only a data structure.

Name	Dimensions	Example in this meeting	Shape
scalar	0	the bias of one neuron	a single number
vector	1	one pixel; the weights of one neuron	`(3,)`
matrix	2	the points `X`; the weights `W` of a layer	`(n, 3)`, `(K, 3)`
volume	3	a colour image	`(height, width, 3)`
(no special name)	4	a batch of colour images	`(30, 96, 96, 3)`

`torch.from_numpy` makes a tensor from a NumPy array and keeps its type; `torch.tensor` makes one from a list, with PyTorch's default type.

Code: Step 10, the layer of Act 1 again, and three tensors — cells `image-number`, `tensors`

```
IMAGE = 7        # replace 0 with your group's number, from 1 to 30

assert 1 <= IMAGE <= 30, "Type your group's number in place of the 0."

images = pd.read_csv(IMAGES + "images.csv", index_col="number")
picture_info = images.loc[IMAGE]
image = read_image(picture_info["file"])
height, width = image.shape[0], image.shape[1]

X = image.reshape(-1, 3)                 # the points: one row for each pixel
n, m = X.shape
K = 5
C = reference_colours(X, K)              # the reference colours
W = C.copy()                             # the weights of the layer: one row for each neuron
b = -0.5 * (C ** 2).sum(axis=1)          # the biases of the layer: one number for each neuron
S = X @ W.T + b                          # the scores: the line of Act 1

print(f"Image {IMAGE}: {picture_info['title']} ({picture_info['creator']})")
print("NumPy arrays.  X:", X.shape, "  W:", W.shape, "  b:", b.shape, "  S:", S.shape)
```

OUTPUT

```
Image 7: The Bedroom (Vincent van Gogh)
NumPy arrays.  X: (320640, 3)   W: (5, 3)   b: (5,)   S: (320640, 5)
```

```
w_t = torch.tensor([0.299, 0.587, 0.114])        # the weights of the grey neuron, from a list
X_t = torch.from_numpy(X)                        # the points, from the NumPy array X
image_t = torch.from_numpy(image)                # the image, from the NumPy array image

for name, tensor in [("w_t", w_t), ("X_t", X_t), ("image_t", image_t)]:
    print(f"{name:8s} dimensions: {tensor.ndim}   shape: {tuple(tensor.shape)}   type of its numbers:
{tensor.dtype}")
```

OUTPUT

```
w_t      dimensions: 1   shape: (3,)   type of its numbers: torch.float32
X_t      dimensions: 2   shape: (320640, 3)   type of its numbers: torch.float64
image_t  dimensions: 3   shape: (501, 640, 3)   type of its numbers: torch.float64
```

2.2 The type of the numbers Step 10

NumPy stores a decimal number as `float64`, with 8 bytes and about 16 significant decimal digits. PyTorch's default is `float32`, with 4 bytes and about 7 digits. Neural networks are usually computed in `float32` or in shorter types, which take less memory and are faster on a GPU. PyTorch does not multiply a `float64` tensor by a `float32` one: `X_t @ w_t` raises an error until `X_t.float()` converts the points. The points then take 3.8 MB in place of 7.7.

Code: Step 10, two types, and the conversion — cell `dtype`

```
try:
    X_t @ w_t
except RuntimeError as error:
    print("X_t @ w_t fails:", error)

X_t = X_t.float()                                # the same numbers, as float32
megabytes = X_t.element_size() * X_t.nelement() / 1e6
print("after X_t = X_t.float(), the type of its numbers is", X_t.dtype)
print(f"memory of the points: {X.nbytes / 1e6:.1f} MB as float64, {megabytes:.1f} MB as float32")

s_t = X_t @ w_t                                  # one neuron, many points: the line of Act 1
print("shape of X_t @ w_t:", tuple(s_t.shape))
```

OUTPUT

```
X_t @ w_t fails: addmv input tensors must have the same dtype, but got Float, Double, and Float
after X_t = X_t.float(), the type of its numbers is torch.float32
memory of the points: 7.7 MB as float64, 3.8 MB as float32
shape of X_t @ w_t: (320640,)
```

2.3 The layer in PyTorch, and the softmax translated Step 11

The line `X_t @ W_t.T + b_t` is the line of Act 1, unchanged. Its scores agree with NumPy's to within `1.8e-07`, not exactly, because the tensors hold `float32` numbers: rounding a number to `float32` can change it by a relative amount of up to $2^{-24} \approx 6 \times 10^{-8}$.

	NumPy	PyTorch
an array from a list	`np.array([1.0, 2.0])`	`torch.tensor([1.0, 2.0])`
the layer	`X @ W.T + b`	`X @ W.T + b`
a function of every entry	`np.exp(S)`	`torch.exp(S)`
along the rows	`axis=1`, `keepdims=True`	`dim=1`, `keepdim=True` (NumPy's names are also accepted)
the largest entry of each row	`S.max(axis=1)`, an array	`S.max(dim=1)`, with `.values` and `.indices`
default decimal type	`float64`	`float32`
where the numbers are	the main memory	a **device**: the CPU or a GPU

Step 11 asks an AI assistant to translate `softmax_rows` into `softmax_rows_torch`: a tensor in, a tensor of the same shape out, no loop, no NumPy, and not PyTorch's own softmax. The translation has three differences: `torch.exp` for `np.exp`; `dim` and `keepdim` for `axis` and `keepdims`; and `.values` after `S.max(dim=1, keepdim=True)`, which returns the largest values together with their positions. Without `keepdim=True`, the largest entries would have shape `(n,)`, and broadcasting would align them with the K columns rather than the n rows: an error when $n \neq K$, and wrong numbers when $n = K$. The function may not use NumPy because NumPy cannot compute with a tensor stored on a GPU. The check compares it with `torch.softmax(S, dim=1)`.

Code: Step 11, the layer in PyTorch, the translated softmax, and its check — cells `layer-torch`, `paste-softmax-torch`, `run-check-softmax-torch`, `repaint-torch`

```
W_t = torch.from_numpy(W).float()        # the weights of the layer, as a float32 tensor
b_t = torch.from_numpy(b).float()        # the biases of the layer

S_t = X_t @ W_t.T + b_t                  # the line of Act 1, unchanged
print("shape of S_t:", tuple(S_t.shape))
print(f"largest difference from the NumPy scores S: {np.abs(S_t.numpy() - S).max():.1e}")

winner_t = S_t.argmax(dim=1)                    # NumPy: S.argmax(axis=1)
largest_t = S_t.max(dim=1, keepdim=True)        # NumPy: S.max(axis=1, keepdims=True)
print("shape of S_t.argmax(dim=1):", tuple(winner_t.shape))
print("S_t.max(dim=1, keepdim=True) returns two tensors: .values of shape", tuple(largest_t.values.shape),
      "and .indices of shape", tuple(largest_t.indices.shape))
print("torch.sigmoid of the first row of scores:", torch.sigmoid(S_t[0]))
```

OUTPUT

```
shape of S_t: (320640, 5)
largest difference from the NumPy scores S: 1.8e-07
shape of S_t.argmax(dim=1): (320640,)
S_t.max(dim=1, keepdim=True) returns two tensors: .values of shape (320640, 1) and .indices of
shape (320640, 1)
torch.sigmoid of the first row of scores: tensor([0.5732, 0.5721, 0.5803, 0.5693, 0.5720])
```

```python
def softmax_rows_torch(S):
    shifted = S - S.max(dim=1, keepdim=True).values  # the largest entry of every row becomes 0
    E = torch.exp(shifted)
    return E / E.sum(dim=1, keepdim=True)
```

```python
assert "softmax_rows_torch" in globals(), "Paste the function softmax_rows_torch into the cell above and
run that cell."
assert not has_loop(softmax_rows_torch), "This function must use operations on whole tensors: no loop."
assert not mentions(softmax_rows_torch, ("softmax(", "Softmax", "scipy", "sklearn")), (
    "This function must compute the softmax itself: not torch.softmax, and no softmax function from
another library.")
assert not mentions(softmax_rows_torch, ("np.", "numpy")), (
    "This function must compute with PyTorch only: no NumPy function, and no conversion to a NumPy array.
"
    "NumPy cannot compute with a tensor that is on a GPU.")

test = torch.tensor([[-1.0, 0.0, 2.0, 1.0],
                     [0.0, 0.0, 0.0, 0.0],
                     [1000.0, 1003.0, 1001.0, 1002.0]])
try:
    result = softmax_rows_torch(test.clone())
except TypeError:
    raise AssertionError(
        "A tensor was combined with something that is not a tensor. In PyTorch, S.max(dim=1, keepdim=True)
returns "
        "two things, the largest values and their positions: take its .values.") from None
except RuntimeError:
    raise AssertionError(
        "The shapes inside softmax_rows_torch do not fit. The largest entry and the sum of every row must
keep "
        "the shape (n, 1): use keepdim=True.") from None
assert isinstance(result, torch.Tensor), "softmax_rows_torch must return a PyTorch tensor."
assert result.shape == test.shape, f"softmax_rows_torch returned a tensor of shape {tuple(result.shape)};
it must be {tuple(test.shape)}."
assert not torch.isnan(result).any(), (
    "softmax_rows_torch returns nan (not a number) for the row of very large scores. "
    "Subtract the largest entry of each row before the exponential.")
assert torch.allclose(result.sum(dim=1), torch.ones(3)), (
    "The rows of the result do not add to 1. The largest entry and the sum must be taken along each row:
dim=1.")
assert torch.allclose(result, torch.softmax(test, dim=1), atol=1e-5), (
    f"The result is not the softmax of the rows. For the scores -1, 0, 2, 1 the function gives
{result[0].tolist()}; "
    "the right values are 0.03, 0.09, 0.64, 0.24.")
print("Correct: softmax_rows_torch agrees with PyTorch's own torch.softmax(S, dim=1), also for very large
scores.")
```

OUTPUT

```
Correct: softmax_rows_torch agrees with PyTorch's own torch.softmax(S, dim=1), also for very
large scores.
```

```
# the paints of the five neurons: replace these rows with the paints of your group
PAINT = np.array([[0.10, 0.07, 0.30],
                  [0.80, 0.10, 0.35],
                  [0.98, 0.55, 0.15],
                  [0.99, 0.85, 0.40],
                  [1.00, 1.00, 0.92]])
T = 0.02                                  # the temperature of the softmax

PAINT_t = torch.from_numpy(PAINT).float()
Prob_t = softmax_rows_torch((X_t @ W_t.T + b_t) / T)     # scores, then probabilities
new_t = Prob_t @ PAINT_t                                  # probabilities, then colours
print("shape of new_t:", tuple(new_t.shape), "  type of its numbers:", new_t.dtype)
show([image, new_t.reshape(height, width, 3).numpy()], ["the image", "repainted with PyTorch"])
```

OUTPUT

```
shape of new_t: (320640, 3)   type of its numbers: torch.float32
```

2.4 PyTorch's own layer Step 12

`torch.nn.Linear(3, K)` makes a layer of K neurons for points with 3 coordinates, with random weights and biases. Its weights have shape `(K, 3)`, one neuron in each row as in `W`, and PyTorch's documentation writes the layer as $y = xA^T + b$: the line of Act 1. With the weights and biases of Act 1 copied into it, its scores equal those of `X_t @ W_t.T + b_t`. Training a network requires, at each step, the rate at which the network's answers change with each weight, called a **gradient**. PyTorch can compute gradients; `torch.no_grad()` states that this cell needs none.

Code: Step 12, torch.nn.Linear — cell `linear`

```
linear = torch.nn.Linear(3, K)           # a layer of K neurons for points with 3 coordinates
print("shape of linear.weight:", tuple(linear.weight.shape))
print("shape of linear.bias:  ", tuple(linear.bias.shape))

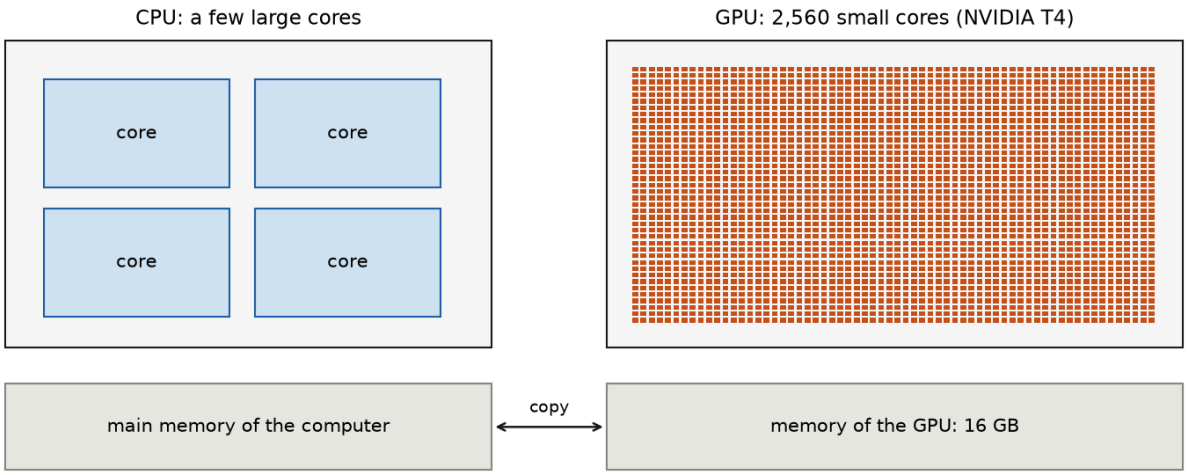
with torch.no_grad():                    # this cell needs no gradients
    linear.weight.copy_(W_t)             # the weights of Act 1 go into the layer
    linear.bias.copy_(b_t)               # and so do the biases
    S_linear = linear(X_t)               # the layer, applied to all the points

print("shape of linear(X_t):", tuple(S_linear.shape))
difference = float((S_linear - S_t).abs().max())
print(f"largest difference between linear(X_t) and X_t @ W_t.T + b_t: {difference:.1e}")
```

```
OUTPUT
shape of linear.weight: (5, 3)
shape of linear.bias:   (5,)
shape of linear(X_t): (320640, 5)
largest difference between linear(X_t) and X_t @ W_t.T + b_t: 0.0e+00
```

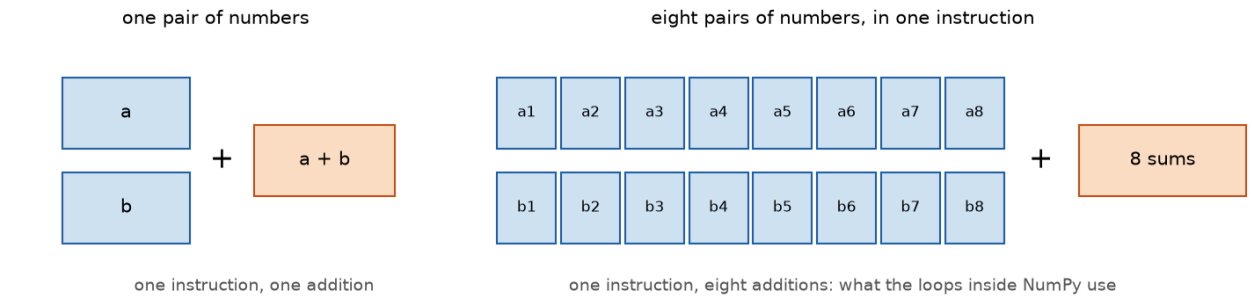
2.5 A second processor: the GPU Step 13

The **CPU**, the central processing unit, runs the programs of a computer. It has a few **cores**: 2 on Colab's free machines, 4 to 16 in a laptop, more than 100 in a large server. Each core runs any program, one instruction after another. A **GPU**, a graphics processing unit, was built to compute the colours of the pixels of a screen: one short computation repeated for millions of pixels that do not depend on each other. It therefore has thousands of simpler cores that carry out the same instruction on different numbers at the same time, and a memory of its own. According to its data sheet, the NVIDIA T4 that Colab lends has 2,560 cores, 16 GB of memory, a peak of 8.1 trillion `float32` operations a second, and a power rating of 70 watts.



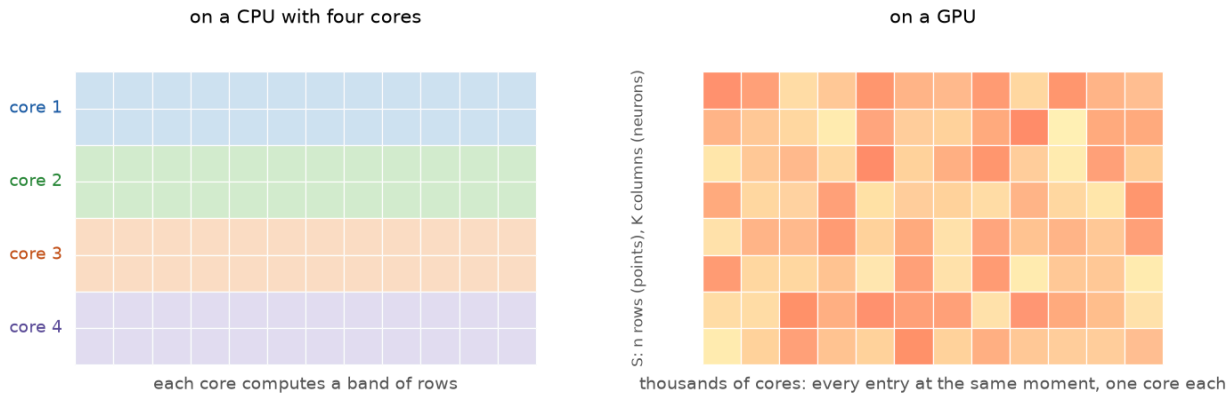
Two processors, each computing with the numbers of its own memory.

Three features make a CPU core fast at the arithmetic of this course. One instruction can add or multiply several pairs of numbers at once, eight on many processors; the loops inside NumPy and PyTorch use such instructions, while the Python interpreter handles one number at a time. A small, fast memory in the core, the **cache**, makes numbers that stay in it quick to read; the notes of meeting 12 measure this. And several cores can share one large product.



One instruction, one addition; one instruction, eight additions.

A product of matrices has the form of the pixels of a picture: each entry of S is a sum of m products that depends on no other entry. The work can therefore be divided among any number of cores. A CPU gives each of its cores a band of rows; a GPU computes thousands of entries at the same time. This is why the layers of neural networks are computed on GPUs.



One product divided among the cores of a CPU and of a GPU.

Code: Step 13, the processors of this session — cell `gpu-facts`

```
print("logical cores of the CPU that this session can use:", os.cpu_count())
if GPU:
    gpu = torch.cuda.get_device_properties(0)
    print("the GPU:", gpu.name)
    print(f"memory of the GPU: {gpu.total_memory / 2 ** 30:.1f} GB")
else:
    print("This session has no GPU. Read these numbers in the notebook of a member whose session has
one.")
```

OUTPUT

```
logical cores of the CPU that this session can use: 32
This session has no GPU. Read these numbers in the notebook of a member whose session has one.
```

2.6 A tensor lives on a device Step 14

PyTorch names the GPU `"cuda"`, after CUDA, the system with which NVIDIA's GPUs are programmed. `X_t.to("cuda")` copies the numbers of a tensor into the GPU's memory and returns the copy. An operation needs all of its tensors on one device, so `X_g @ W_t.T`, with `X_g` on the GPU and `W_t` in the main memory, raises an error. With all three tensors copied, the line of the scores does not change, and the GPU carries it out. `.cpu()` copies a result back. The machine that built these notes has no GPU, so the output below is the notebook's message for a session without one. In a T4 session the cell prints the device of each tensor, the error, and the largest difference between the scores computed by the two processors.

The copies pass through the connection between the two memories, and they take time. Part 4 of the page CPU and GPU lists when a GPU does not pay: data that is copied there and back for one short operation, data that does not fit in the GPU's memory, and steps that each need the result of the step before. For a large product whose data stays on the GPU, it pays. In a T4 session, the full notebook measures the copy of one 4,096 × 4,096 matrix of `float32` numbers, 67 MB, beside the product at that size; no class post reported that line.

Code: Step 14, tensors on two devices — cell `device`

```
print("X_t is on the device:", X_t.device)
if GPU:
    X_g = X_t.to("cuda")                # copies of the three tensors, in the memory of the GPU
    W_g = W_t.to("cuda")
    b_g = b_t.to("cuda")
    print("X_g is on the device:", X_g.device)

    try:
        X_g @ W_t.T                     # X_g is on the GPU and W_t is on the CPU
    except RuntimeError as error:
        print("X_g @ W_t.T fails:", error)

    S_g = X_g @ W_g.T + b_g             # the line of Act 1, carried out by the GPU
    print("S_g is on the device:", S_g.device)
    difference = float((S_g.cpu() - S_t).abs().max())
    print(f"largest difference between the scores of the GPU and of the CPU: {difference:.1e}")
else:
    print("This session has no GPU, so every tensor stays in the memory of the CPU.")
```

OUTPUT

```
X_t is on the device: cpu
This session has no GPU, so every tensor stays in the memory of the CPU.
```

2.7 Measuring the product on both processors Step 15

The layer on an image is a small product: `W` has 5 rows and 3 columns.
`time_product(size, device)` measures `X @ W.T + b` with $n = m = K =$ `size`, on random `float32` numbers made on the device itself, so that no copy is timed. At size 4,096 the product takes 4096^3,

about 69 billion, multiplications. Two details are particular to a GPU. `torch.cuda.synchronize()` waits until the GPU has finished; without it, the clock would measure the handing over of the work, not the work. And the first use of a device takes extra time, so one small product runs first and is not measured.

Code: Step 15, the timer — cell `time-product`

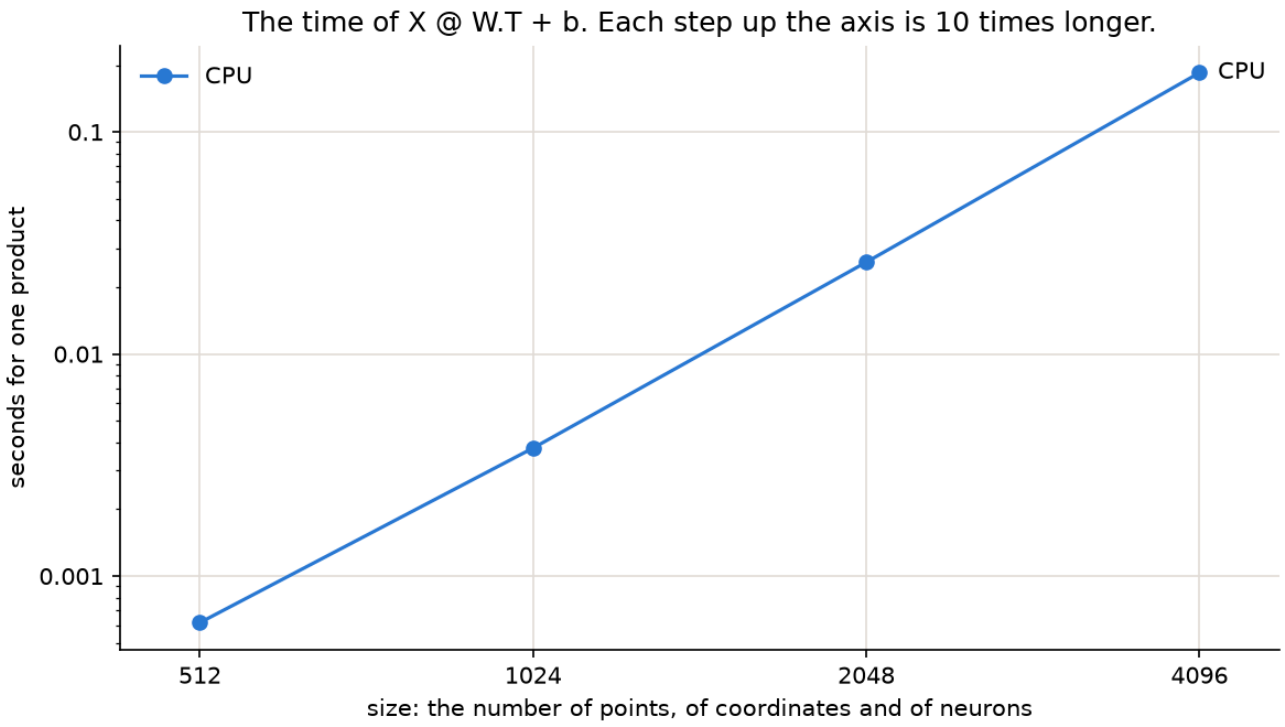
```
def finish(device):
    """Wait until the device has finished its work. A GPU may still be working when PyTorch goes on."""
    if device == "cuda":
        torch.cuda.synchronize()


def time_product(size, device):
    """The average time, in seconds, of X @ W.T + b on the device, with n = m = K = size."""
    X = torch.rand(size, size, device=device)        # random float32 numbers, made on the device
    W = torch.rand(size, size, device=device)
    b = torch.rand(size, device=device)
    repeats = max(1, min(200, 8_000_000_000 // size ** 3))     # many repeats when the product is small
    finish(device)
    start = time.perf_counter()
    for _ in range(repeats):
        S = X @ W.T + b
    finish(device)
    return (time.perf_counter() - start) / repeats


def warm_up(device):
    """One small product that is not measured."""
    time_product(64, device)


print("time_product and warm_up are defined.")
```

On the machine that built these notes, a CPU with 32 logical processors, the product at size 4,096 took 0.18 seconds. Both axes of the chart are logarithmic: each size is twice the one before, and each mark on the vertical axis is ten times the one below. On such axes a time that grows as a power of the size, $t \approx c \cdot \text{size}^{p}$, is a straight line of slope p; an exponential time would curve upward.



The CPU of the machine that built these notes. A session with a GPU draws a second line.

Code: Step 15, the measurement and the chart

cells `measure-devices`, `timing-chart`

```
sizes = [512, 1024, 2048, 4096]
devices = ["cpu", "cuda"] if GPU else ["cpu"]
seconds = {}
for device in devices:
    warm_up(device)
    seconds[device] = [time_product(size, device) for size in sizes]

gpu_name = torch.cuda.get_device_name(0) if GPU else "none"
print(f"X @ W.T + b with n = m = K = size, in seconds. CPU: {os.cpu_count()} logical cores. GPU:
{gpu_name}.")
print(f"{'size':>6s}{'multiplications':>18s}{'CPU':>12s}" + (f"{'GPU':>12s}{'CPU / GPU':>12s}" if GPU else
""))
for i, size in enumerate(sizes):
    line = f"{size:6d}{size ** 3:18,d}{seconds['cpu'][i]:12.6f}"
    if GPU:
        line += f"{seconds['cuda'][i]:12.6f}{seconds['cpu'][i] / seconds['cuda'][i]:12.1f}"
    print(line)

if GPU:
    X_cpu = torch.rand(4096, 4096)
    finish("cuda")
    start = time.perf_counter()
    X_gpu = X_cpu.to("cuda")
    finish("cuda")
    t_move = time.perf_counter() - start
    megabytes = X_cpu.element_size() * X_cpu.nelement() / 1e6
    print(f"\nCopying one matrix of size 4096 ({megabytes:.0f} MB) to the GPU takes {t_move:.4f} seconds;
```

```
"
          f"the product at that size takes the GPU {seconds['cuda'][-1]:.4f} seconds.")
```

OUTPUT

```
X @ W.T + b with n = m = K = size, in seconds. CPU: 32 logical cores. GPU: none.
  size   multiplications         CPU
   512       134,217,728    0.000617
  1024     1,073,741,824    0.003772
  2048     8,589,934,592    0.025927
  4096    68,719,476,736    0.184849
```

```
fig, ax = plt.subplots(figsize=(8, 4.6))
ax.plot(sizes, seconds["cpu"], marker="o", color="#2a78d6", label="CPU")
ax.annotate("CPU", (sizes[-1], seconds["cpu"][-1]), xytext=(8, 0), textcoords="offset points",
va="center")
if GPU:
    ax.plot(sizes, seconds["cuda"], marker="s", linestyle="--", color="#eb6834", label="GPU")
    ax.annotate("GPU", (sizes[-1], seconds["cuda"][-1]), xytext=(8, 0), textcoords="offset points",
va="center")
ax.set_xscale("log", base=2)
ax.set_yscale("log")
ax.set_xticks(sizes)
ax.set_xticklabels([str(size) for size in sizes])
ax.yaxis.set_major_formatter(plt.FuncFormatter(lambda value, position: f"{value:g}"))
ax.margins(x=0.08)
ax.set_xlabel("size: the number of points, of coordinates and of neurons")
ax.set_ylabel("seconds for one product")
ax.set_title("The time of X @ W.T + b. Each step up the axis is 10 times longer.")
ax.grid(True, which="major", color="#e1e0d9", linewidth=0.8)
ax.spines[["top", "right"]].set_visible(False)
ax.legend(frameon=False)
fig.tight_layout()
plt.show()

last, first = len(sizes) - 1, 0
print("Line for the discussion board:")
if GPU:
    print(f"GPU {gpu_name}, CPU with {os.cpu_count()} logical cores. Size 4096: CPU {seconds['cpu']
[last]:.3f} s, "
          f"GPU {seconds['cuda'][last]:.4f} s, so the GPU is {seconds['cpu'][last] / seconds['cuda']
[last]:.0f} times as fast. "
          f"Size {sizes[first]}: CPU {seconds['cpu'][first] * 1e3:.1f} milliseconds, GPU {seconds['cuda']
[first] * 1e3:.2f} milliseconds.")
else:
    print(f"No GPU in this session. CPU with {os.cpu_count()} logical cores. Size 4096: {seconds['cpu']
[last]:.3f} s. "
          f"Size {sizes[first]}: {seconds['cpu'][first] * 1e3:.1f} milliseconds.")
```

OUTPUT

```
Line for the discussion board:
No GPU in this session. CPU with 32 logical cores. Size 4096: 0.185 s. Size 512: 0.6
milliseconds.
```

Class posts.

On the board, 20 posts came from sessions with a T4, 19 of them with a two-core CPU, and 3 from sessions without a GPU. At size 4,096 the T4 took 0.0486 to 0.0490 seconds in every session but one, which took 0.0631. The two-core CPUs took 1.06 to 1.95 seconds, with one at 3.97, and the sessions without a GPU 1.90 to 2.40 seconds. The T4 was 22 to 40 times as fast as the two-core CPU of its session, with a median of 31. The session with the slow T4 also had the slowest CPU, and a factor of 63. In the one session whose CPU had 8 logical cores, the CPU took 0.342 seconds and the factor was 7.

The factor was larger at size 4,096 than at size 512 in 18 of the 19 two-core sessions, and its median at 512 was 16. The one post that gave its whole table shows where the growth stops: 24 at size 512, then 34, 33 and 35 at sizes 1,024 to 4,096. One post explained the growth by the cost of copying the data to the GPU; the timing of Step 15 makes its tensors on the device, so it includes no copy.

Of the posts that answered the second question, eight gave a growth factor between 6.9 and 8 for the CPU's time from size 2,048 to 4,096, and most of the others gave 6 to 12. Doubling n, m and K multiplies the number of multiplications by $2^3 = 8$; the notes of meeting 12 explain why a measured time can differ from that count. One post called the growth exponential: it is a power of the size, which appears as a straight line on the chart's axes. One post from a session without a GPU quoted an expected factor of 100 to 200; no session on the board measured more than 63. Report only what your own session measured.

After the meeting

The record of a group's work in this meeting is its two posts on the discussion board, each beginning with the names of all the members of the group: the repainted image of Act 1 with the printed line and a sentence on the temperature, and the chart of Act 2 with the printed line and the two answers. Nothing is uploaded. If a post is missing, the paragraphs marked **Post** in Step 7 of Act 1 and Step 15 of Act 2 say what to post.

Review Quiz 11 draws on the full notebooks. Six short questions ask for a score by hand, the shapes of a layer, the sigmoid and the softmax of a matrix, NumPy expressions in PyTorch, tensors on two devices, and a large product on a GPU. Two questions of a few sentences follow one pixel through the two lines of Step 7, with the nearest-colour argument of idea 6 and the effect of the temperature; and compare the NumPy and PyTorch versions of the softmax, with the group's measurements of Step 15 and the cost of a copy. The module of this meeting also holds **Homework 1**, *Vectorization*.

built from the notebooks of meeting 11, run for group 7, on 2026-10-14